Projet Master IPSA

Modelling of the transport of a chemical species in a fluid

BARROTTA Emanuela
MAZURIE Corentin

Contents

INTRODUCTION AND MOTIVATION 2

METHODOLOGY AND ORGANIZATION 4

PHYSICAL MODELLING OF THE TRANSPORT OF A CHEMICAL SPECIES IN A FLUID IN 1D 6

THE CONVECTION-DIFFUSION EQUATION 6
SCALE ANALYSIS WITH PÉCLET NUMBER 6
ANALYTICAL SOLUTION WITH THE METHOD OF CHARACTERISTICS 7
DISCRETIZATION AND NUMERICAL STABILITY 9
NUMERICAL RESULTS AND DISCUSSIONS 10

PHYSICAL MODELING OF TRANSPORT IN 2D 17

PHYSICAL MODELING AND ANALYTICAL SOLUTION IN 2D 17
DISCRETIZATION OF THE MODEL 17
RESULTS AND COMMENTS 20

THE ISSUE WITH OUR CURRENT MODEL 25

STUDY OF THE BROWNIAN MOTION 25

HOW TO SIMULATE THE MOVEMENT OF A PARTICLE IN A FLUID? 25
THE CHOICE OF THE PROBABILITY LAW 25
PYTHON SIMULATION OF A WIENER PROCESS 25

THE STOCHASTIC TRANSPORT EQUATION 27

NEW DEFINITION OF THE EQUATION AND SOLUTION 27
THE EULER-MARUYAMA METHOD 27
RESULTS AND DISCUSSION 28

CONCLUSION AND PERSPECTIVES 32

ANNEXES 33

PYTHON CODE 33
1D SPACE-TIME REPRESENTATION OF TEST CASES 47

SOURCES 52

Introduction and motivation

The purpose of the project was to model the behaviour of a chemical species in a fluid. We compared different models to better understand the limits and the qualities of each.

To study the evolution in time of the concentration of a species, convection and diffusion must be considered. Convection is the transport of the species by the fluid's velocity field; diffusion is the consequence of the concentration gradient, which means that the species tends to move from a region of higher concentration to one of lower concentration.

Our analysis began by considering the convection-diffusion equation in the incompressible case, in which we neglected the diffusion term. Given boundary and initial conditions, we computed the theoretical solution of the partial differential equation (PDE) using the method of characteristics, to provide an analytical solution for comparison with the numerical models.

For the numerical solution, we used the Finite Differences Method for spatial discretization and various methods for temporal discretization, to compare the stability and accuracy of each. The developed schemes were implemented in Python.

These steps were firstly applied to some 1D domain problems: CO_2 in water or air and the case of oil in water, to consider a problem with immiscible species. Only in the latter case we chose not to neglect the diffusion term. In the 1D tests, we compared the stability and accuracy of the explicit Euler method with the implicit Euler method and the Crank-Nicolson method.

We then applied the same procedure on a 2D domain problem and compared the results obtained using the method of characteristics with those derived from three numerical methods: upwind FDM, centered FDM and RK4 algorithm.

A more realistic model of transport was achieved by including a stochastic term in the transport equation to take into account the random movement of particles at a microscopic scale, known as Brownian motion. We used a Gaussian distribution centered at zero to model the displacement of particles with respect to the previous position. The new problem was exclusively studied using the Euler-Maruyama numerical method, which allowed to obtain a solution to compare with the non-stochastic model.

Part 1:
Methodology and organization of this work

Methodology and organization

For this project we followed a structured methodology to study the transport equation and the various aspects of the project. We started by doing in-depth research about the transport equation and its analytical solutions using the method of characteristics. We could then validate the results we got for the equation using tools like WolframAlpha and other sources. Then, we were interested in the numerical methods to solve partial differential equations, including a detailed study of numerical schemes, their stability and boundary conditions. Another important aspect of our project was the study of the stochastic transport equation and stochastic processes such as the Brownian motion, as well as the numerical schemes used for solving such problems.

To effectively manage the project, we outlined three main axes of work. The first axis focused on a detailed examination of the transport equation in 1D, covering its definition, analytical solution, numerical solution with scheme comparisons, and Python implementation. The second axis extended this study to the 2D realm, mirroring the structure of the first axis. The third axis focused on the stochastic transport equation, involving a comprehensive study of Brownian motion, the equation's definition, numerical solutions, and subsequent tests and discussions. Looking ahead, we explored potential avenues for further research, pondering how our findings could pave the way for deeper investigations or practical applications in related fields.

The work was divided as follows between the members of the group:

BARROTTA Emanuela	MAZURIE Corentin
Definition of the convection-diffusion equation Scale analysis with the Péclet number Numerical solutions in 1D Scheme stability comparison in 1D Biphasic case	Method of characteristics in 1D and 2D Python implementation in 1D and 2D Numerical solutions in 2D Scheme stability comparison in 2D Results discussion in 1D and 2D Study of the Brownian motion Definition of the stochastic equation Python implementation of the SPDE using the Euler-Maruyama method

Part 2:
Physical modelling and mathematical analysis in 1D and 2D

Physical modelling of the transport of a chemical species in a fluid in 1D

The convection-diffusion equation

We first consider the **convection-diffusion equation** in the **incompressible** case:

$$\boxed{\frac{\partial u}{\partial t} + c \cdot \nabla u - \nu \Delta u = f}$$

With:

- u: The studied variable, in our case the concentration of a chemical species in a fluid
- c: Velocity of the fluid
- $\frac{\partial u}{\partial t}$: Partial derivative of u with respect to time
- ∇u: Divergence of u
- Δu: Laplacian of u
- ν: Kinematic viscosity of the fluid
- f: Source term

This equation is defined in a limited space (or closed set) Ω[1] and for $t \in [0, t_f]$. In this equation, $c \cdot \nabla u$ corresponds to the **convection** term and $\nu \Delta u$ is the **diffusion** term. This equation describes the temporal evolution of the **passive** scalar u. We say that a scalar is passive when it doesn't influence the motion of the fluid, i.e. the velocity field doesn't depend on u.

Scale analysis with Péclet number

As its name suggests, the convection-diffusion equation models the presence of two different physical phenomena: convection and diffusion. We can use the **Péclet number** to determine which one of these phenomena prevails.

The Péclet number is defined as follows:

$$\boxed{Pe = \frac{|c|L}{2\nu}}$$

Where:

- c: Velocity of the fluid
- L: Characteristic length of the space domain
- ν: Kinematic viscosity of the fluid

Three cases can appear:
If $Pe \sim 1$, convection and diffusion effects are comparable.

if $Pe \ll 1$, the diffusion effects are dominant with respect to the convection effects.

if $Pe \gg 1$, the convection effects are dominant.

In our study, we will consider cases with $c \sim 1\ m/s$ and $L \sim 1m$. Moreover, the kinematic viscosity ν depends on the fluid and, according to the following table, we can assume for ν a value of 10^{-4} m^2/s or lower.

[1] In 1D Ω = [0, L], in 2D $\Omega = [0, L_1] * [0, L_2]$

Fluid	Dynamic Viscosity (Ns/m^2)	Kinematic Viscosity (m^2/s)
Water	1.00×10^{-3}	1.00×10^{-6}
Sea Water	1.07×10^{-3}	1.04×10^{-6}
Mercury	1.56×10^{-3}	1.15×10^{-7}
Kerosene	1.92×10^{-3}	2.39×10^{-4}
Air	1.80×10^{-5}	1.494×10^{-5}
Carbon Dioxide	1.48×10^{-5}	0.804×10^{-5}

Fig 1: Kinematic viscosity of some fluids at 20°C

For instance, $\nu = 1.00 * 10^{-6}\ m^2/s$ for water at 20°C.

This means that in our case it is always $Pe \gg 1$, meaning that the **diffusion term is negligible**. This simplifies our equation, and we obtain the **transport equation**:

$$\boxed{\frac{\partial u}{\partial t} + c \cdot \nabla u = f}$$

Analytical solution with the method of characteristics

In one dimension, using the cartesian coordinates system, the studied problem can be written like this:

$$\begin{cases} \dfrac{\partial u(t,x)}{\partial t} + c \cdot \dfrac{\partial u(t,x)}{\partial x} = f(t,x) \qquad t \in [0, t_f] \quad x \in [0, L] \\ u(0,x) = u_0(x) \qquad if\ x - ct > 0 \\ u(t,x) = g(t) \qquad if\ x - ct < 0 \end{cases}$$

We choose to solve the problem with the **method of characteristics**: the idea is that we will look for curves, also called **characteristics**, along which the Partial Differential Equation (PDE) can be reduced to an Ordinary Differential Equation (ODE) easier to solve.

To transform the PDE in an ODE, we start by supposing $x = x(s)$ and $t = t(s)$.

We have $u(t,x) = u(s)$ and $f(t,x) = f(s)$.

We compute the derivative of u with respect to s and get:

$$\frac{du}{ds} = \frac{\partial u}{\partial t} t' + \frac{\partial u}{\partial t} x'$$

By imposing $x'(s) = c$ and $t'(s) = 1$ we have $x(s) = cs + x_0$ and $t(s) = s + t_0$.

We then impose $t_0 = 0$, hence $t = s$ and $x = ct + x_0$.

Thus:

$$\boxed{\frac{du}{ds} = \frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = f(t,x) = f(s)}$$

This is a **first order equation**; the general solution is given by:

$$\boxed{u(s) = u_h(s) + u_p(s)}$$

Where $u_h(s)$ is the solution of the homogeneous equation and $u_p(s)$ is a particular solution.

Homogeneous equation

If there is no source term (i.e. $f = 0$) we have:

$$\boxed{\frac{du}{ds} = 0}$$

Hence:

$$u(s) = u(0) = u\big(t(s=0), x(s=0)\big) = u_0(x_0) = u_0(x - ct)$$

The solution of the homogeneous equation is:

$$\boxed{u_h(s) = u_h(t,x) = u_0(x - ct)}$$

Particular solution

We have

$$\boxed{\frac{du}{ds} = f(s)}$$

We know that $t = s$ and $x = ct + x_0$, hence:

$$\frac{du}{dt} = f\big(t, x(t)\big) = f(t)$$

So, a particular solution for this equation is:

$$\boxed{u_p(s) = u_p(t,x) = \int_0^t f(t,x)dt = \int_0^t f(t, ct + x_0)dt}$$

General solution

Finally, **the general solution of the transport equation in 1D can be written as**:

$$\boxed{u(t,x) = \begin{cases} u_0(x - ct) + \int_0^t f(t,x)dt \;\; if\; x - ct > 0 \\ g(t - {}^{x}/_{c})\; else \end{cases}}$$

Discretization and numerical stability

Numerical method and used notations

For the discretization we will use the **finite difference method** and the **upwind scheme**. Suppose we discretize time and space as follows:

- $[0, t_f]$ is divided in N parts with a time-step of $\Delta t = \frac{t_f}{N}$
- $[0, L]$ is divided in M parts with a space-step of $\Delta x = \frac{L}{M}$
- $u(t,x)$ becomes u_i^n, $n \in [0,N]$, $i \in [0,M]$
- $f(t,x)$ becomes f_i^n, $n \in [0,N]$, $i \in [0,M]$
- $\frac{\partial u}{\partial t}$ becomes $\frac{u_i^{n+1}-u_i^n}{\Delta t}$
- $\frac{\partial u}{\partial x}$ becomes $\frac{u_i^n-u_{i-1}^n}{\Delta x}$ (or $\frac{u_{i+1}^n-u_i^n}{\Delta x}$ or $\frac{u_{i+1}^n-u_{i-1}^n}{2\Delta x}$)
- $U^n = \begin{pmatrix} u_0^n \\ \cdots \\ u_M^n \end{pmatrix}$
- $F^n = \begin{pmatrix} f_0^n \\ \cdots \\ f_M^n \end{pmatrix}$

The discretized equation is therefore:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + c\frac{u_i^n - u_{i-1}^n}{\Delta x} = f_i^n$$

Hence the following numerical scheme:

$$\boxed{u_i^{n+1} = u_i^n + \frac{c\Delta t}{\Delta x}(u_{i-1}^n - u_i^n) + \Delta t * f_i^n}$$

For faster computation, we can notice that the scheme can be put into a matrix form:

$$\boxed{U^{n+1} = AU^n + \Delta t * F^n}$$

Where:

$$\boxed{A = I_M + \frac{c\Delta t}{\Delta x}\begin{pmatrix} -1 & 0 & \cdots & \cdots & 0 \\ 1 & \ddots & \ddots & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & 1 & -1 \end{pmatrix}}$$

For the scheme to be numerically stable it needs to fulfill the **Courant–Friedrichs–Lewy condition**.

The CFL condition is defined as follows, for a time-explicit scheme, in 1D:

$$\boxed{\frac{c\Delta t}{\Delta x} \leq 1}$$

This condition ensures that the information does not propagate too far during a single time step, preventing instability in the numerical solution. If the CFL condition is violated, the numerical solution may exhibit unphysical oscillations or diverge from the true solution.

For the time discretization we consider two alternative methods and compare the stability with respect to the explicit method used above.

If we discretize using the **implicit Euler method**, the numerical scheme can be written as follows:

$$\boxed{u_i^{(n+1)} = u_i^n + \frac{c\Delta t}{\Delta x}\left(u_{(i-1)}^{(n+1)} - u_i^{(n+1)}\right) + \Delta t * f_i^{(n+1)}}$$

The implicit methods don't have any restriction on the time step: the implicit Euler method is **unconditionally stable**. It is still a **first order approximation** method, so we are not adding any more precision with respect to the explicit Euler method.

If we discretize using the **Crank-Nicolson method**, the numerical scheme can be written as follows:

$$\boxed{u_i^{(n+1)} = u_i^n + \frac{\Delta t}{2}\left(\frac{c}{\Delta x}\left(u_{(i-1)}^n - u_i^n\right) + f_i^n + \frac{c}{\Delta x}\left(u_{(i-1)}^{(n+1)} - u_i^{(n+1)}\right) + f_i^{(n+1)}\right)}$$

The advantage of this method is not only that it is **unconditionally stable** but also that it is a **second order approximation** method. To effectively have a second order precision in the obtained solution, also the space discretization must be of the second order.

Numerical results and discussions

We coded the numerical solution of the transport equation in Python using different numerical schemes presented earlier. To verify the validity of our schemes we compared them to the analytical solution of the equation. The advantage of using a numerical scheme is to obtain good approximations of the solution to an equation without having to solve it. In general, we use numerical schemes for equations that we cannot solve analytically, Navier-Stokes equations for example.

The code is divided into six files:

- Parameters.py contains the parameters of the simulation.
- Conditions.py contains the conditions of the system (i.e. $f(t,x)$ and $g(t)$).
- Analytical.py allows us to compute the analytical solution of the system.
- Numerical.py allows us to compute the numerical solution of the system.
- Display.py allows us to display and save graphs and animations.
- Main.py calls functions from the presented files.

The external libraries we used were *NumPy* and *Matplotlib*, respectively for matrix computation and graph display.

For all the following tests we will have $L = 1\text{m}$, $t_f = 1\text{s}$, $M = 100$, $N \leq c * M$. With these parameters, the CFL condition is verified. Indeed, with $\Delta t = \frac{t_f}{N} \leq \frac{1}{cM}$ and $\Delta x = \frac{L}{M} = \frac{1}{M}$ we have:

$$\frac{c\Delta t}{\Delta x} \leq \frac{c\frac{1}{cM}}{\frac{1}{M}} \leq 1$$

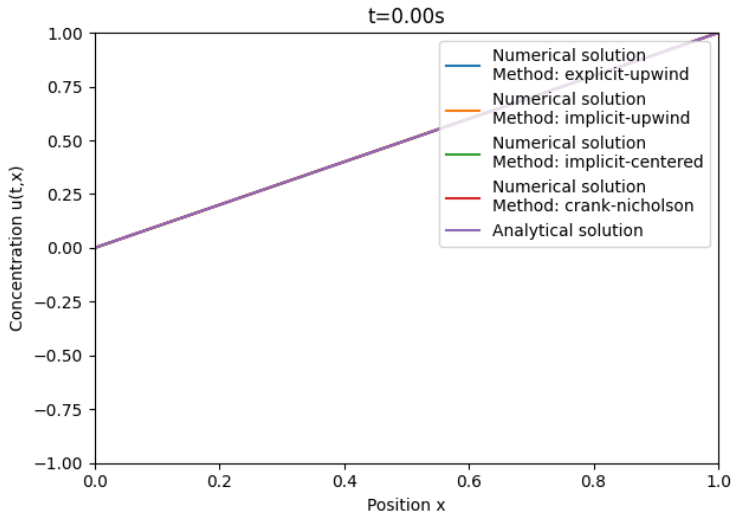
But before computing the concentration we check if the condition is verified anyway, using the function *check_cfl.* The code used developed for this report is available in the annexes.

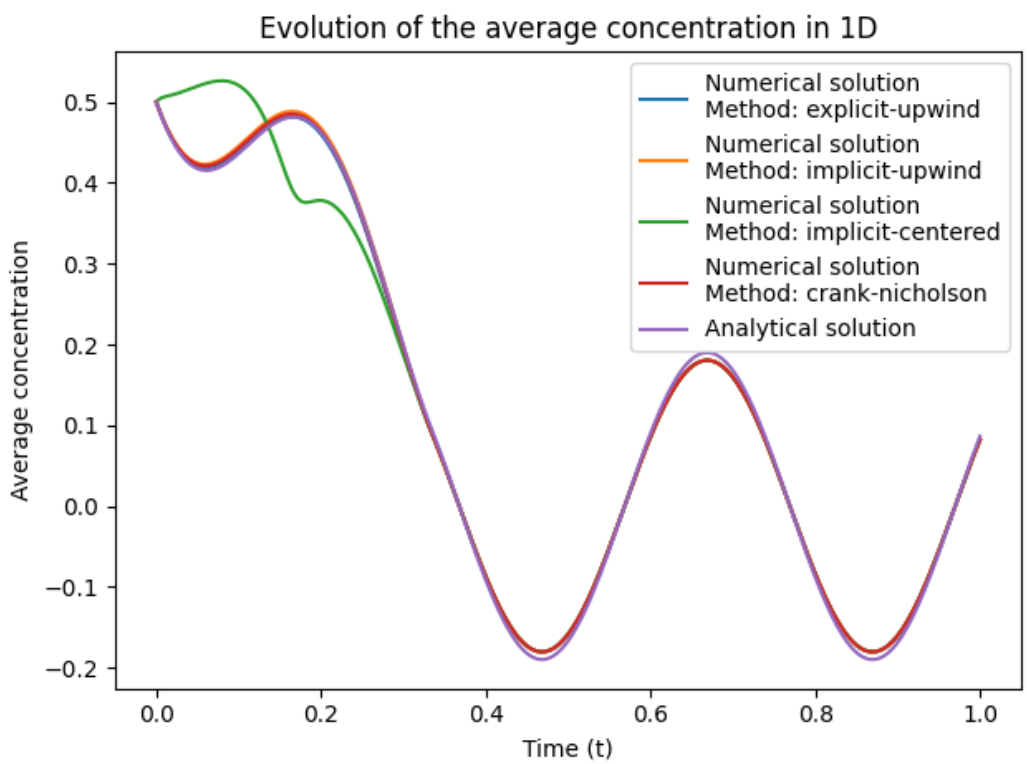
Test case 1

The purpose is to compute the evolution of the concentration of a species (CO_2 for example) in water in a 1D domain. Given the velocity of the fluid c, to solve the system of equations we need to define the initial conditions $u_0(x)$ and the boundary conditions g(t). In this case, the initial conditions are described as a linear function of x, while the boundary conditions are periodic. The source term is null.

$$\begin{cases} u(0,x) = u_0(x) = x \; if \; x > ct \\ u(t,x) = g(t) = \sin(5\pi t) \; if \; x < ct \\ c = 3 \\ f(t,x) = 0 \end{cases}$$

We get the following results:



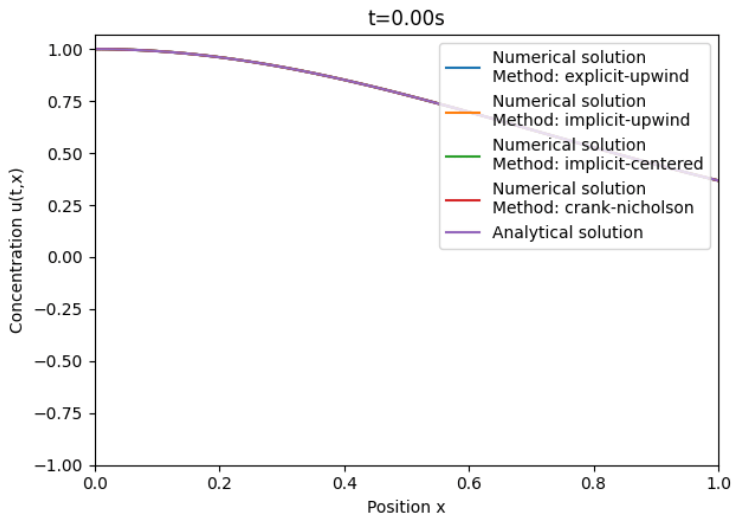


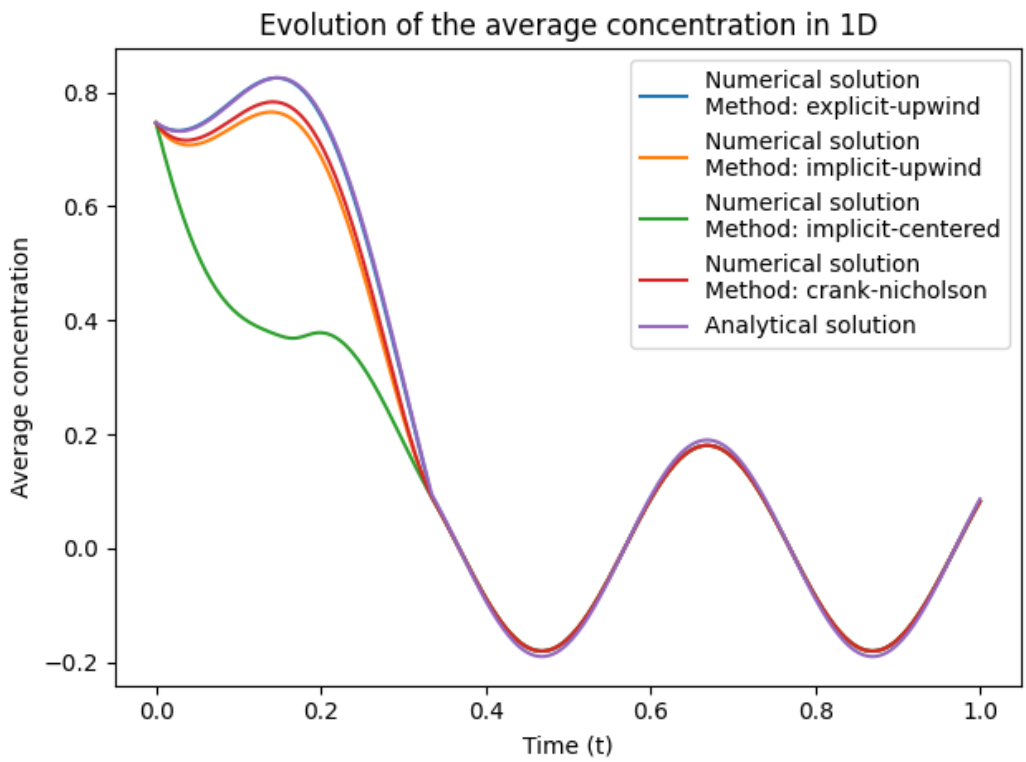
Test case 2

In the second test the boundary conditions are still periodic and the source term is null. The initial conditions instead vary as an exponential function.

$$\begin{cases} u_0(x) = \exp(-x^2) \\ g(t) = \sin(5\pi t) \\ c = 3 \\ f(t,x) = 0 \end{cases}$$

We get the following results:



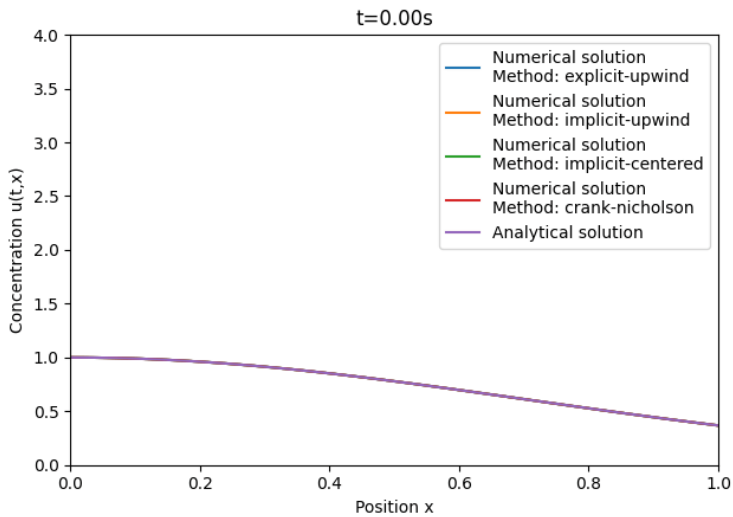


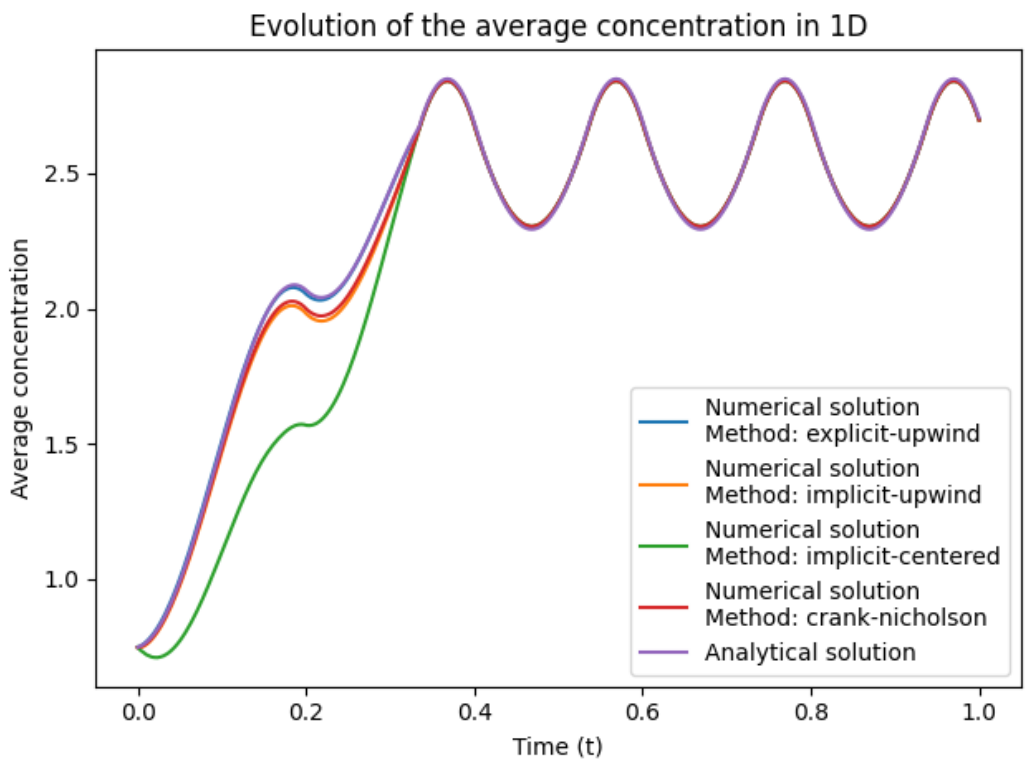
Test case 3

In the third test we introduce the absolute value in the boundary conditions and the source term is a constant in time and space, different from zero.

$$\begin{cases} u_0(x) = \exp(-x^2) \\ g(t) = 4|\sin(5\,\pi\,t)| \\ c = 3 \\ f(t,x) = 2 \end{cases}$$

We get the following results:



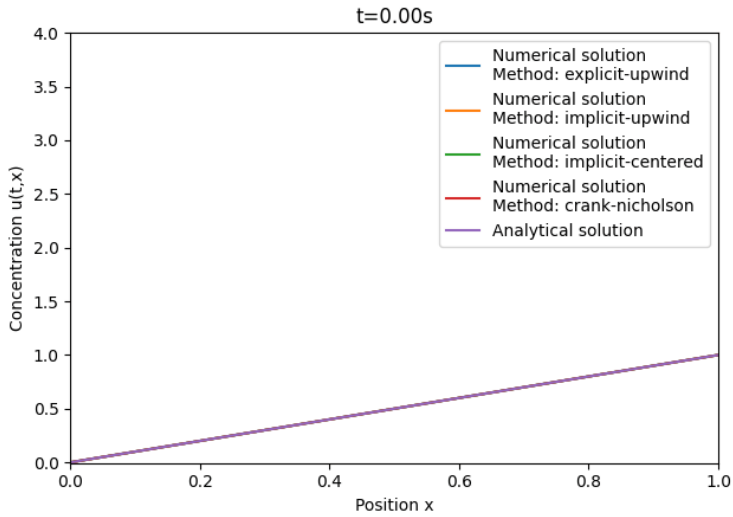


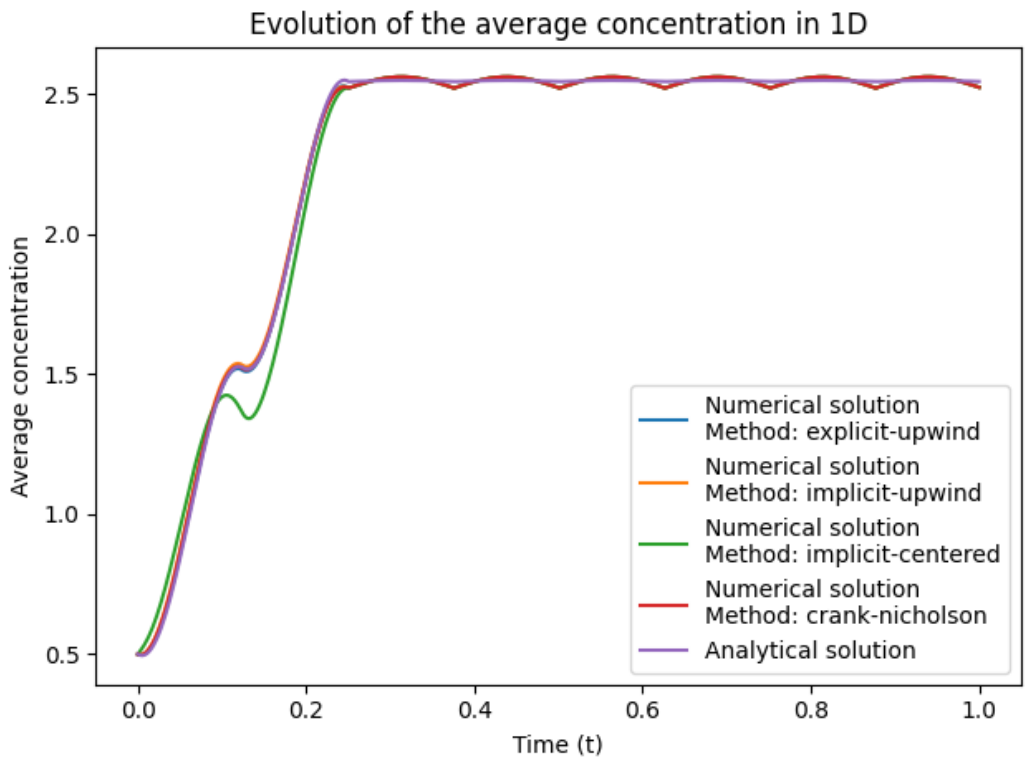
Test case 4: Transport of CO2 in the air

The aim is to study the concentration of CO_2 above the town of Ivry-sur-Seine during an hour. We consider the city to extend over 5 km, with a factory at the southern entrance of the city, which uses blast furnaces four times per hour, causing a significant emission of CO_2. The problem can be modelized considering the transport equation and the following conditions:

$$\begin{cases} u_0(x) = x \\ g(t) = 4|\sin(8\pi\, t)| \\ c = 4 \\ f(t,x) = 2 \end{cases}$$

We get the following results:





Test case 5: Oil in water

The purpose is to compute the evolution of the concentration of oil in water in a 1D domain. Since the pollutant is not miscible in the fluid, the flow is composed of two different fluid phases, thus we have the presence of an interface. Each phase is supposed to be present with a certain volume fraction, constant in space. We choose a continuous phases model, i.e. each phase occupies a different region of the space, like in a free surface flow. There is no mass transfer between the phases and there is not tension at the interface.

The problem is described by the following equation:

$$\boxed{\frac{\partial u(x,t)}{\partial \mathrm{t}} + \mathrm{c}\frac{\partial u(x,t)}{\partial \mathrm{x}} - \nu\frac{\partial^2 u(x,t)}{\partial \mathrm{t}^2} = \mathrm{f(x,t)}}$$

Where $\nu = 1.12 * 10^{-8} m^2/s$ is the diffusivity of oil in water at 20°C.

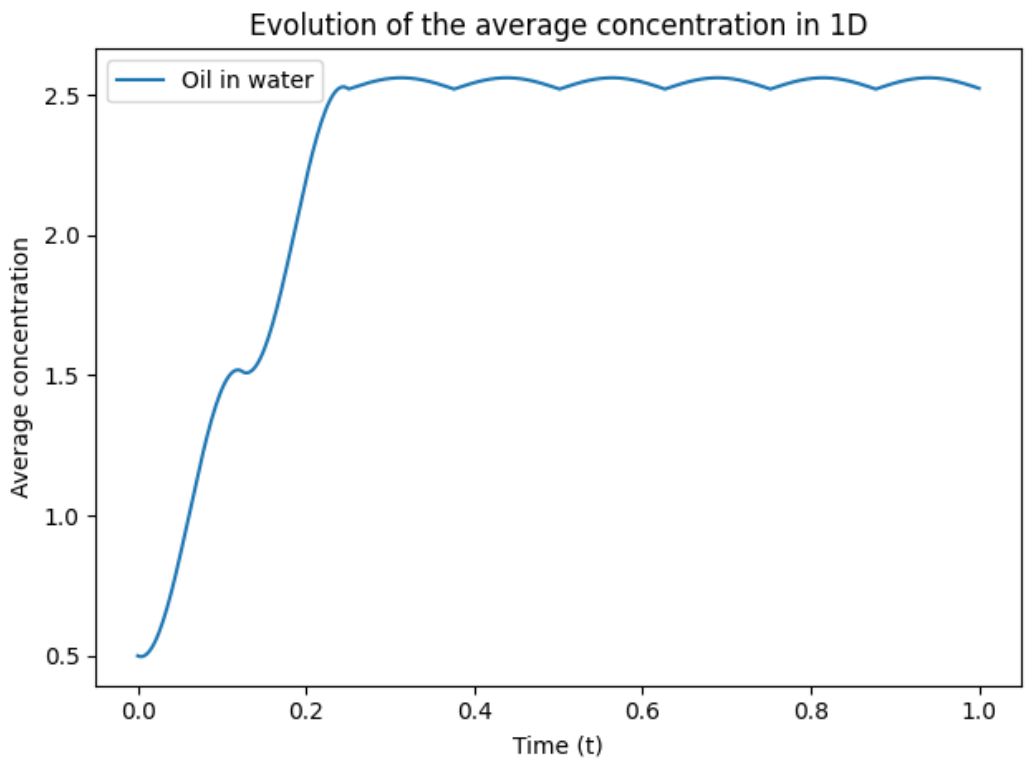
For the discretization we use the finite difference method and the upwind scheme. The discretized equation is:

$$\boxed{\frac{u_i^{n+1} - u_i^n}{\Delta t} + \frac{c}{\Delta x}(u_i^n - u_{i-1}^n) - \nu\frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{(\Delta x)^2} = f_i^n}$$

Hence the following numerical scheme:

$$\boxed{u_i^{n+1} = u_i^n + \frac{c\Delta t}{\Delta x}(u_{i-1}^n - u_i^n) + \frac{\nu\Delta t}{(\Delta x)^2}(u_{i-1}^n - 2u_i^n + u_{i+1}^n) + \Delta t * f_i^n}$$

Given the velocity of the water c, to solve the system of equations we need to define the initial conditions $u_0(x)$ and the boundary conditions g(t). We use the conditions used for the fourth test and obtain the following results:



Discussion

Based on the graphs presented and available in the annexes, we can conclude that the **explicit upwind scheme is the most accurate** of all the schemes tested, despite its simplicity. The main source of error comes when $x \approx ct$, leading to a significative difference between the analytical and numerical solutions. This is probably caused by the **non-continuity** between u_0 and g when $x = ct$.

Even though the 1D case is the easiest we would expect more complex schemes, especially the Crank-Nicholson scheme, to perform better. We thought about two causes for other schemes to perform worse than the explicit upwind scheme: a possible wrong implementation of the schemes or parameters not enough precise for these schemes. However, the main advantage of the other schemes tested is that they are **unconditionally stable**, while the explicit upwind scheme needs to fulfill the CFL condition.

For the case with oil in water we get nearly identical results to those we got for the fourth test. This is coherent with the hypothesis we made in the beginning, i.e. the diffusion term is negligible, so, even if we take it into account the result does not change much.

Physical modeling of transport in 2D

Physical modeling and analytical solution in 2D

The transport equation problem can be modelized using the following system:

$$\begin{cases} \frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} + c\frac{\partial u}{\partial y} = f(t,x,y)\ t \in [0,t_f]\ x \in [0,L]\ y \in [0,L] \\ u(0,x,y) = u_0(x,y)\ if\ \|(x,y)\|_2^2 > ct \\ u(t,x,y) = g(t)\ if\ \|(x,y)\|_2^2 < ct \end{cases}$$

The method of characteristics can be generalized in n dimensions. For the 2D case, the solution is:

$$\boxed{u(t,x,y) = \begin{cases} u_0(x-ct, y-ct) + \int_0^t f(t,x,y)dt\ if\ \|(x,y)\|_2^2 - ct > 0 \\ g(t - \|(x,y)\|_2^2/c)\ else \end{cases}}$$

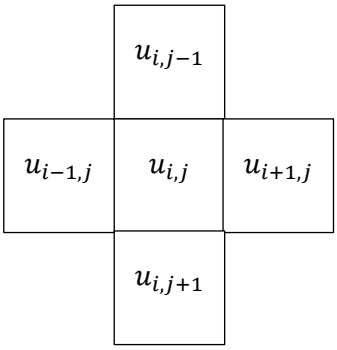
Discretization of the model

Global hypotheses

We suppose that:

- Space, i.e. $[0,L] * [0,L]$, is divided in $N_x * N_y$ elements, such that $x_i = i * \Delta x, \forall i \in [0,N_x]$ with $\Delta x = L/N_x$ and $y_j = j * \Delta y, \forall j \in [0,N_y]$ with $\Delta y = L/N_y$.
- Time, i.e. $[0,t_f]$, is divided in N_t elements, such that $t_n = n * \Delta t, \forall t \in [0,N_t]$ with $\Delta t = 1/N_t$.
- $u(t,x,y)$ becomes $u(t_n,x_i,y_j) = u_{i,j}^n$
- $f(t,x,y)$ becomes $f(t_n,x_i,y_j) = f_{i,j}^n$
- $u_{i,j}^0 = u_0(x_i,y_j)$
- Boundary conditions: $u_{-1,j}^n = u_{i,-1}^n = u_{N_x+1,j}^n = u_{i,N_y+1}^n = 0$
- $N_x = N_y = N$ hence $\Delta x = \Delta y = h$

The discretization looks like this:



The following parts present several numerical schemes we implemented in Python, we will then compare their accuracy and stability.

Finite difference method: upwind scheme

$$\begin{cases} \dfrac{\partial u}{\partial t} \approx \dfrac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t} \\ \dfrac{\partial u}{\partial x} \approx \dfrac{u_{i,j}^{n} - u_{i-1,j}^{n}}{h} \\ \dfrac{\partial u}{\partial y} \approx \dfrac{u_{i,j}^{n} - u_{i,j-1}^{n}}{h} \end{cases}$$

Hence:

$$\frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t} + c\,\frac{u_{i,j}^{n} - u_{i-1,j}^{n}}{h} + c\,\frac{u_{i,j}^{n} - u_{i,j-1}^{n}}{h} = f_{i,j}^{n}$$

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \Delta t * f_{i,j}^{n} - \frac{c\Delta t}{h}\left(2u_{i,j}^{n} - u_{i-1,j}^{n} - u_{i,j-1}^{n}\right)$$

$$\boxed{u_{i,j}^{n+1} = \left(1 - \frac{2c\Delta t}{h}\right)u_{i,j}^{n} + \frac{c\Delta t}{h}\left(u_{i-1,j}^{n} + u_{i,j-1}^{n}\right) + \Delta t * f_{i,j}^{n}}$$

When $i = j = 0$:

$$\boxed{u_{0,0}^{n+1} = \left(1 - \frac{2c\Delta t}{h}\right)u_{0,0}^{n} + \Delta t * f_{0,0}^{n}}$$

When $i = 0, j \neq 0$:

$$\boxed{u_{0,j}^{n+1} = \left(1 - \frac{2c\Delta t}{h}\right)u_{0,j}^{n} + \frac{c\Delta t}{h}u_{0,j-1}^{n} + \Delta t * f_{0,j}^{n}}$$

When $i \neq 0, j = 0$:

$$\boxed{u_{i,0}^{n+1} = \left(1 - \frac{2c\Delta t}{h}\right)u_{i,0}^{n} + \frac{c\Delta t}{h}u_{i-1,j}^{n} + \Delta t * f_{i,0}^{n}}$$

Finite difference method: centered scheme

$$\begin{cases} \dfrac{\partial u}{\partial t} \approx \dfrac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t} \\ \dfrac{\partial u}{\partial x} \approx \dfrac{u_{i+1,j}^{n} - u_{i-1,j}^{n}}{2h} \\ \dfrac{\partial u}{\partial y} \approx \dfrac{u_{i,j+1}^{n} - u_{i,j-1}^{n}}{2h} \end{cases}$$

Hence:

$$\frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t} + c\,\frac{u_{i+1,j}^{n} - u_{i-1,j}^{n}}{2h} + c\,\frac{u_{i,j+1}^{n} - u_{i,j-1}^{n}}{2h} = f_{i,j}^{n}$$

$$\boxed{u_{i,j}^{n+1} = u_{i,j}^{n} + \Delta t * f_{i,j}^{n} - \frac{c\Delta t}{2h}\left(u_{i+1,j}^{n} - u_{i-1,j}^{n} + u_{i,j+1}^{n} - u_{i,j-1}^{n}\right)}$$

Finite difference method + RK4

With the method of characteristics, we can write the PDE as an ODE. So, we had an idea: why don't we try to use an ODE solver to numerically solve the transport equation? The Runge-Kutta algorithm is generally used to solve ODEs of the following form:

$$\begin{cases} y' = f(t, y) \\ y(t_0) = y_0 \end{cases}$$

The approximation of the solution is:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 = f(t_n, y_n) \\ k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\ k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\ k_4 = f(t_n + h, y_n + hk_3) \end{cases}$$

To try to use it for a PDE, we will consider that $f(t, y)$ *is* $f(t, x, y) - c\frac{\partial u}{\partial x} - c\frac{\partial u}{\partial y}$ where $\frac{\partial u}{\partial x}$ and $\frac{\partial u}{\partial y}$ are discretized with the upwind scheme. As a reminder, the code is available in the annexes for more explanation.

Numerical stability

For stability, we can again use the Courant–Friedrichs–Lewy condition for the 2D case:

$$\frac{\mathrm{c}\Delta t}{\Delta x} + \frac{\mathrm{c}\Delta t}{\Delta y} = \frac{2\mathrm{c}\Delta t}{\mathrm{h}} \leq 1$$

Which can also be written as:

$$\boxed{\frac{c\Delta t}{h} \leq \frac{1}{2}}$$

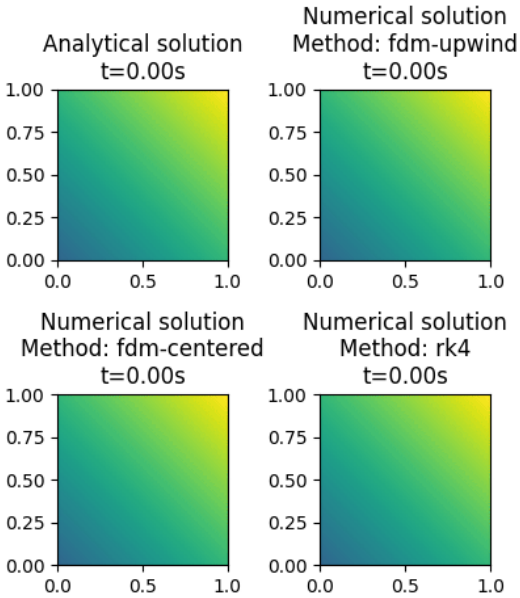
Results and comments

Once again, we coded the 2D numerical schemes presented earlier and compared them to the analytical solution.

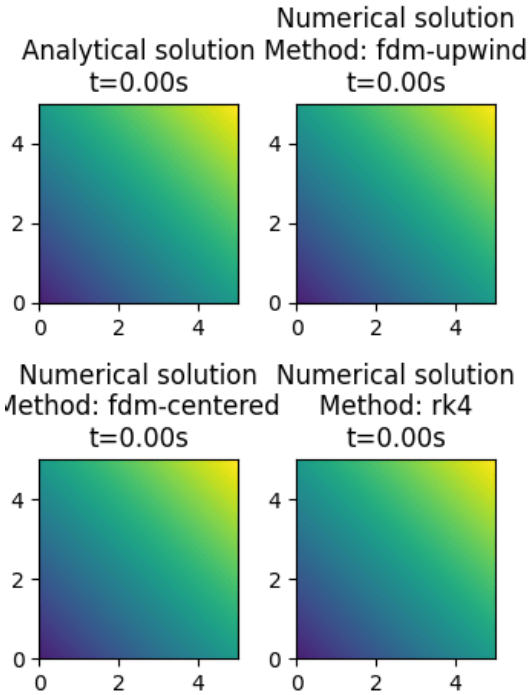
Test case 1

$$\begin{cases} u_0(x,y) = x + y \\ g(t) = \sin(5\pi t) \\ f(t,x,y) = 0 \\ c = 3 \end{cases}$$

We get the following results for $t_f = 1s$ and $L = 1m$:



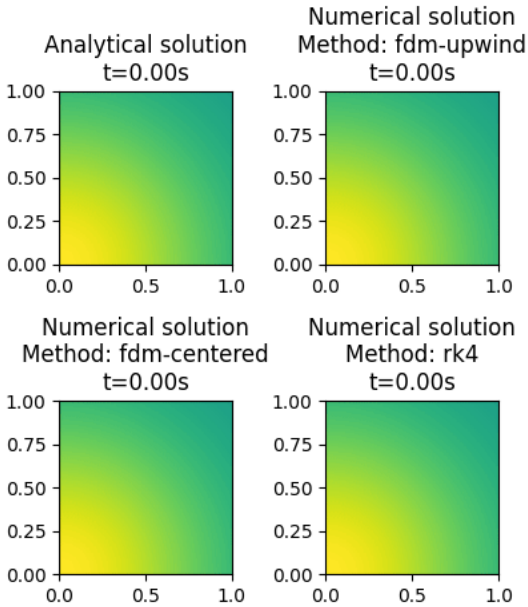
We get the following results for $t_f = 5s$ and $L = 5m$:



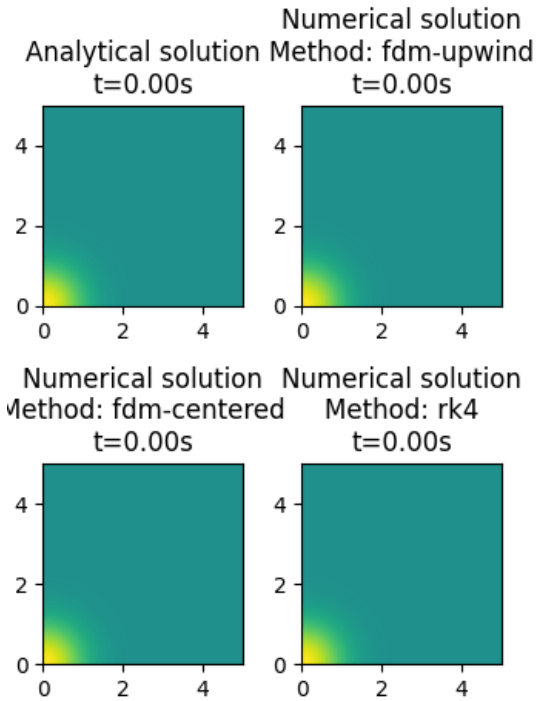
Test case 2

$$\begin{cases} u_0(x,y) = \exp(-x^2 - y^2) \\ g(t) = \sin(5\pi t) \\ f(t,x,y) = 0 \\ c = 3 \end{cases}$$

We get the following results for $t_f = 1s$ and $L = 1m$:



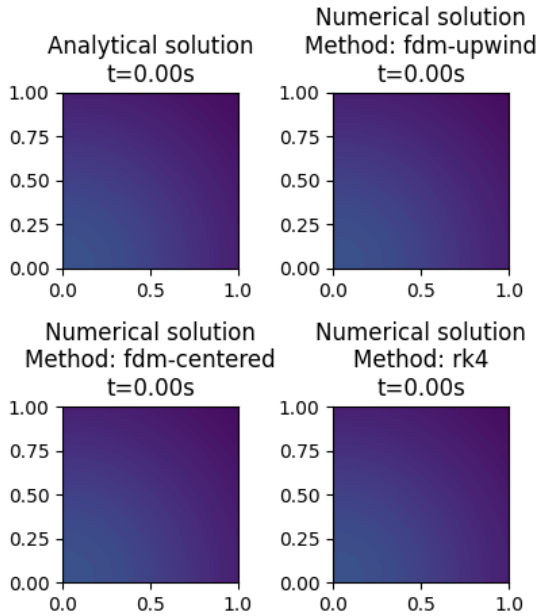
We get the following results for $t_f = 5s$ and $L = 5m$:



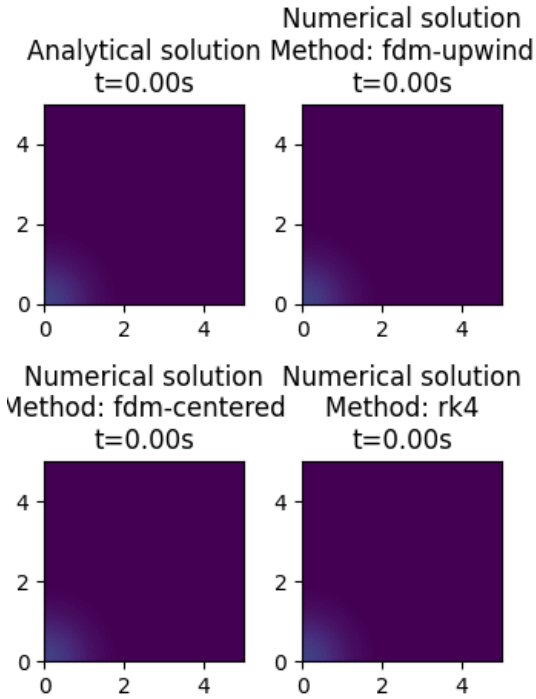
Test case 3

$$\begin{cases} u_0(x,y) = \exp(-x^2 - y^2) \\ g(t) = 4|\sin(5\pi t)| \\ f(t,x,y) = 2 \\ c = 3 \end{cases}$$

We get the following results for $t_f = 1s$ and $L = 1m$:



We get the following results for $t_f = 5s$ and $L = 5m$:



Discussion

There are some **limitations** to the numerical schemes we used when we are outside the common time-space domain, i.e. when $L > 1m$ or $t_f > 1$s. Moreover, the RK4 algorithm and the centered FDM fail to handle the boundaries properly, leading to **unstable results**, as we can see when values which become yellow, indicating a high value increase. This could also be caused by a wrong implementation of these methods. The upwind scheme on the other hand seems stable even at the boundaries but fails to get accurate results when $L > 1$ and $t_f > 1$. We also have some kind of ***diffusion* effect** with the upwind scheme, but it does not reflect the real analytical solution, so it may be related to the numerical method and its simplicity.

Part 3:
Stochastic Partial Differential Equation for the transport a of chemical species in a fluid

The issue with our current model

Let us recall the objective of our subject: we want to simulate the transport of a chemical species in a fluid. To carry out this task, we use the transport equation that we defined earlier. So far, we have obtained interesting results. However, if we were to mention a problem with our equation, which is in fact common to most physical models, it is that it is too **simplistic**. Indeed, we do not consider some factors, for instance friction between particles of the fluid and particles of the chemical species. The transport equation previously presented offers a rather macroscopic and *smooth* vision of the model. If we want to have a more **realistic** version of what is really happening, we need to model the problem on a microscopic scale. At this scale, we can associate a random movement with the particles of the chemical species. This movement is called a **Brownian movement** or Brownian motion.

Study of the Brownian motion

How to simulate the movement of a particle in a fluid?

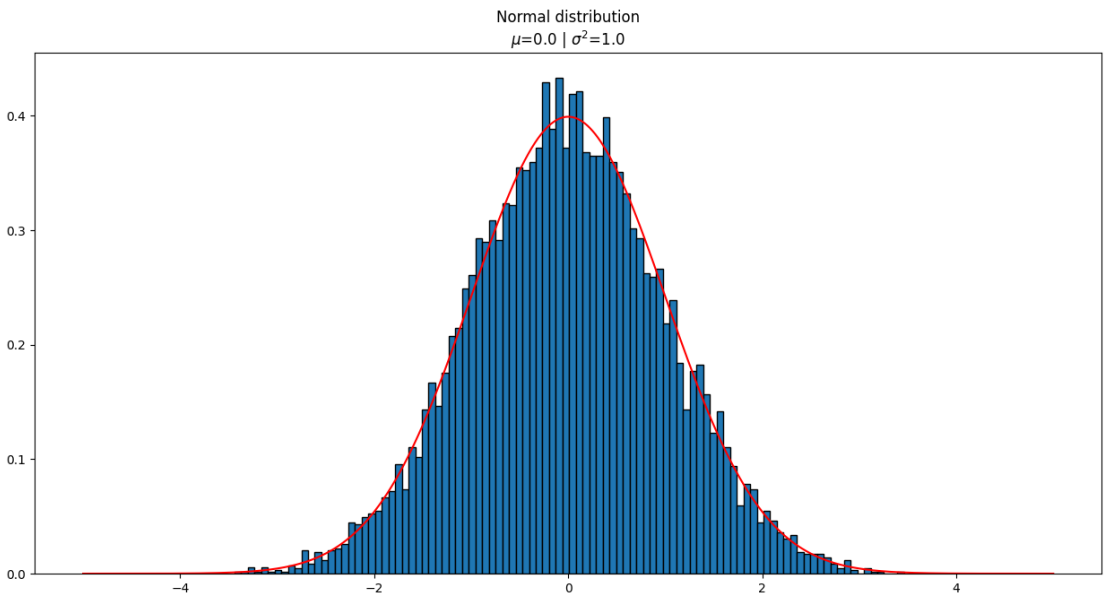
Let's first suppose that our particle is at the position (x, y) at some time t. For the particle to move, we suppose that, at $t + \Delta t$, the particle can move in every direction in space, i.e. along the x and the y axes. The displacement value, called Δx along the x axis and Δy along the y axis, is **independent** of the **previous displacement**, does not depend either on the **direction**, and **follows a normal law** centered at 0 and of variance σ^2, i.e. $\mathcal{N}(0, \sigma^2)$. This type of movement is called a **Wiener process**.

The choice of the probability law

Why did we choose a normal law and not a uniform law for instance? The idea is that, if we observed a particle moving in a fluid, we would see that most of the time it stays close to its previous position, and in some rare cases the displacement of the particle would be higher. This means we need a probability law where the distribution of small values is more common than high values, hence the choice for a gaussian distribution centered around zero.

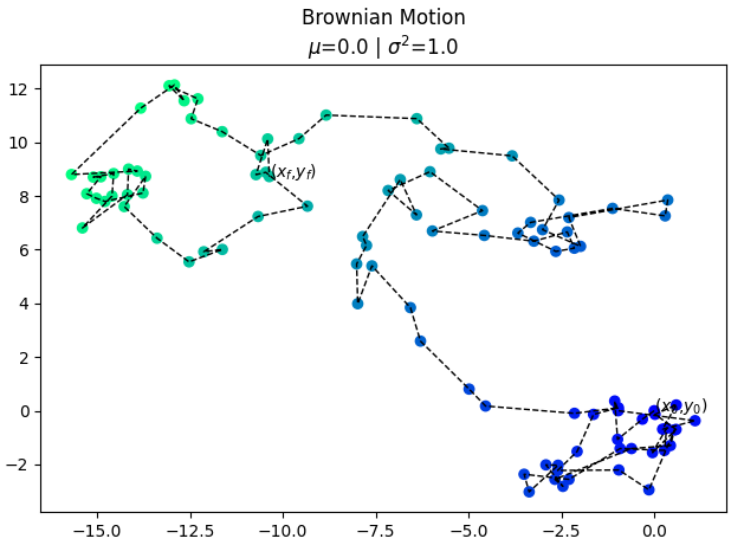
Python simulation of a Wiener process

In Python, we can easily simulate a Wiener process. We will use $\mathcal{N}(0,1)$ as our distribution law for both x and y axes. Below is an example of the normal distribution:

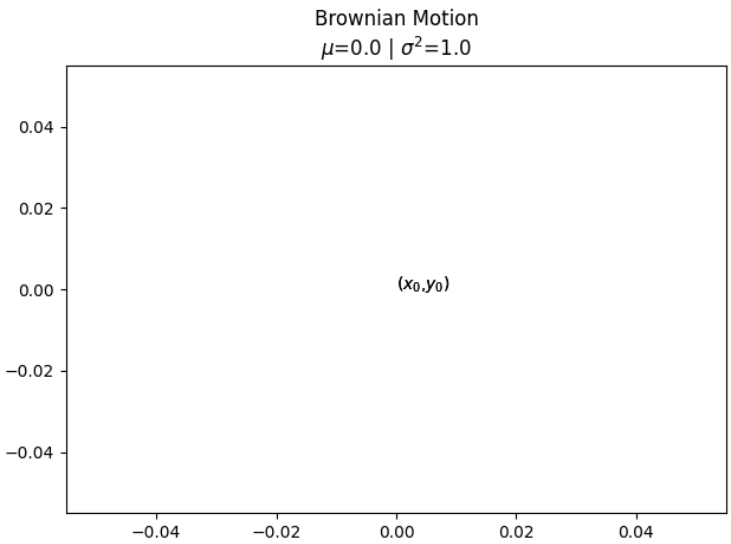


To create the motion, we build a random matrix R of shape $(N-1,2)$, with N the number of steps and two columns which corresponds to the number of axes. The random values of this matrix follow the normal distribution presented above. We then concatenate R with a 1x2 matrix containing the initial position of the particle (x_0, y_0), creating a matrix M of shape $(N, 2)$. Finally, we do the cumulative sum along each column of this matrix M, creating the motion.

Below is plotted an example of Brownian motion for a particle initially at $(x_0, y_0) = (0,0)$:



We can plot the motion using *FuncAnimation* from *Matplotlib*:



The Python code for the Brownian motion can be found in the annex, in the file Wiener.py.

The stochastic transport equation

New definition of the equation and solution

To add the Brownian motion in the transport equation we add a new term called **white noise**. The white noise is defined as the derivative of the Brownian movement over time, i.e. $\frac{\partial W_t}{\partial t}$ where W_t is the Wiener process. The transport equation hence becomes, for the 2D case:

$$\boxed{\frac{\partial u(t,x,y)}{\partial t} + c\frac{\partial u(t,x,y)}{\partial x} + c\frac{\partial u(t,x,y)}{\partial y} + h(t,x,y)\frac{\partial W_t}{\partial t} = f(t,x,y)}$$

Where $h(t,x,y)$ is the **volatility** of the chemical species, it describes the amplitude of the displacement for the particles, we can assimilate it to the variance of the normal distribution. In our case we will consider it constant, i.e. $h(t,x,y) = h = 3$.

We could not find the general analytical solution for this equation and, according to our research, **it does not seem such general solution exists**. So, in the following, we will only be interested in the numerical method we chose, i.e. the **Euler-Maruyama method,** and we will compare the results with the non-stochastic equation.

The Euler-Maruyama method

The Euler-Maruyama method is a numerical technique generally used to approximate stochastic differential equations (SDEs). The general form for a stochastic differential equation is given by:

$$\boxed{dX(t) = a\big(t, X(t)\big)dt + b\big(t, X(t)\big)dW(t)}$$

Where $X(t)$ is the function we want to find, $a(t, X(t))$ is the deterministic part of the equation, $b(t, X(t))$ is the stochastic part of the equation and $dW(t)$ is the derivative of the Wiener process. The numerical solution at each time step (time is divided into interval of width Δt) is given by the following formula:

$$\boxed{X_{n+1} = X_n + a(t_n, X_n)\Delta t + b(t_n X_n)\Delta W_n}$$

Where $\Delta W_n \hookrightarrow \mathcal{N}(0, \Delta t)$.

For partial differential terms, we use the **finite differences method** with **upwind scheme**. Hence, in our case, for the 2D stochastic transport equation, the numerical scheme becomes:

$$\boxed{u_{i,j}^{n+1} = u_{i,j}^n + \frac{c\Delta t}{\Delta x}\left(u_{i-1,j}^n - u_{i,j}^n\right) + \frac{c\Delta t}{\Delta y}\left(u_{i,j-1}^n - u_{i,j}^n\right) + h * \Delta W_n + \Delta t * f_{i,j}^n}$$

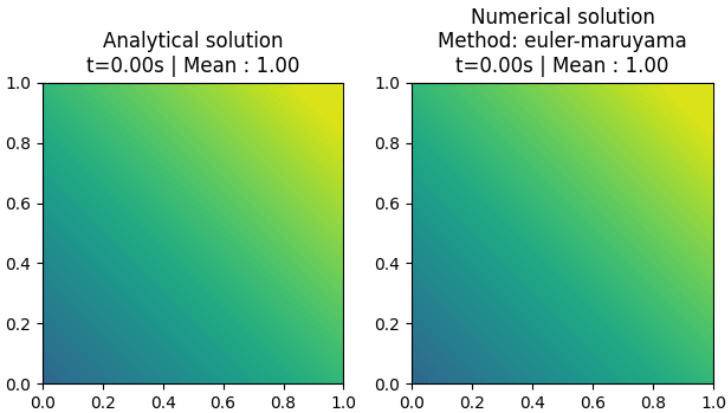
Results and discussion

We programmed in Python the Euler-Maruyama method and tried it on the 2d test cases we presented earlier. We then compared the results obtained with the analytical solution of the non-stochastic transport equation.

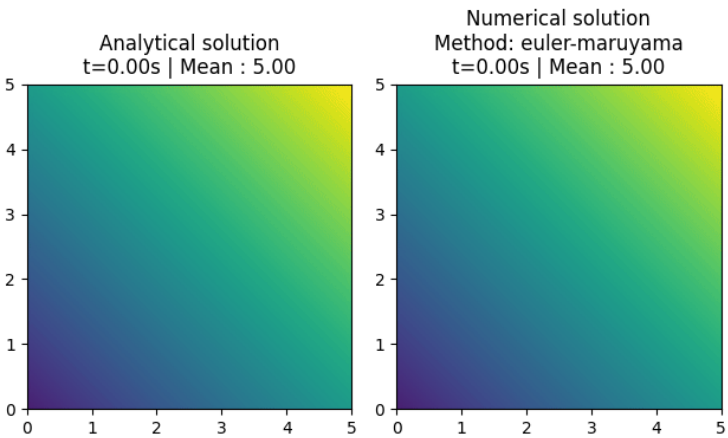
Test case 1

$$\begin{cases} u_0(x,y) = x + y \\ g(t) = \sin(5\pi t) \\ f(t,x,y) = 0 \\ c = 3 \end{cases}$$

For the first test case we get the following results:



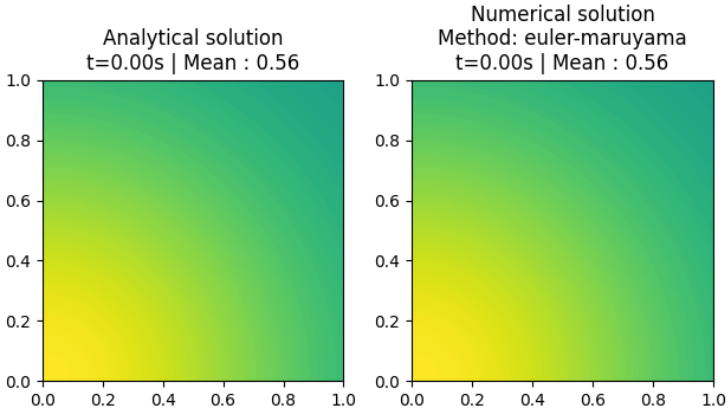
When $t = 5s$ and $L = 5m$ we get:



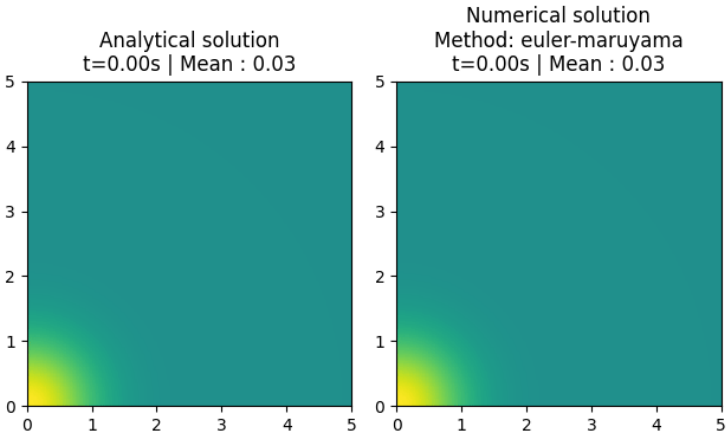
Test case 2

$$\begin{cases} u_0(x,y) = \exp(-x^2 - y^2) \\ g(t) = \sin(5\pi t) \\ f(t,x,y) = 0 \\ c = 3 \end{cases}$$

For the second test case we get the following results:



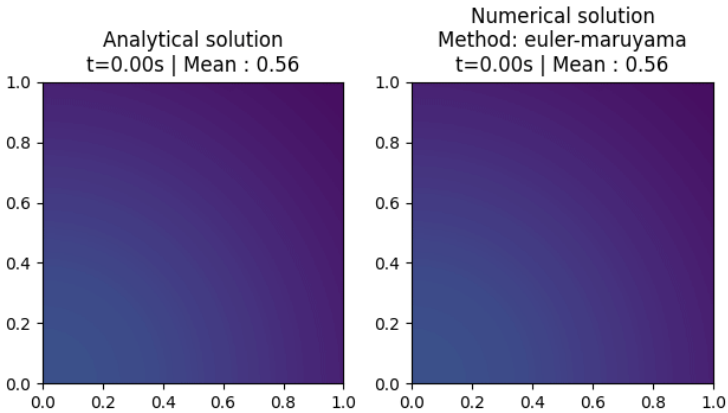
When $t_f = 5s$ and $L = 5m$ we get:



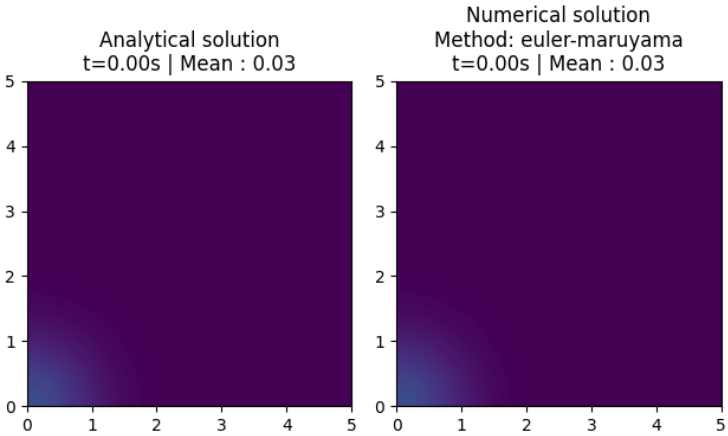
Test case 3

$$\begin{cases} u_0(x,y) = \exp(-x^2 - y^2) \\ g(t) = 4|\sin(5\pi t)| \\ f(t,x,y) = 2 \\ c = 3 \end{cases}$$

For the third test case we get the following results:



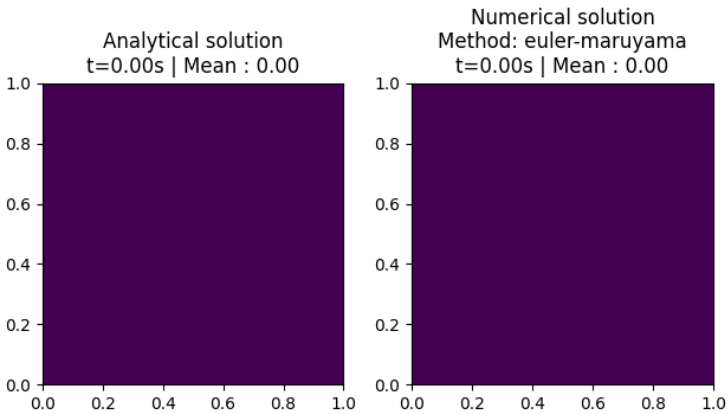
When $t_f = 5s$ and $L = 5m$ we get:



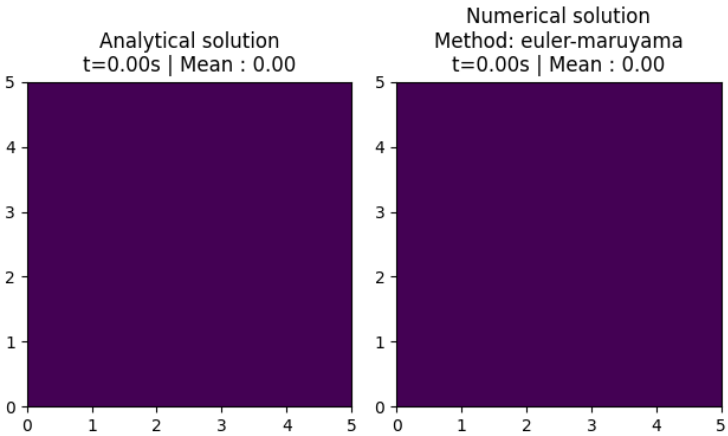
Test case 4: Short input

$$\begin{cases} u_0(x,y) = 0 \\ g(t) = \begin{cases} \sin(10\pi t)\ if\ t < 2 * \Delta t \\ 0\ else \end{cases} \\ f(t,x,y) = 0 \\ c = 3 \end{cases}$$

For this test we get the following results:



When $t_f = 5s$ and $L = 5m$ we get:



Discussion

With the stochastic transport equation, we observe some kind of ***diffusion/transport* effect** of the concentration**.** There are probably **two causes** for this effect: the first one is the **scheme used** since we previously saw a similar phenomenon when using the upwind scheme; the second reason is that it may also be caused by the **Brownian motion**. Intuitively, **we think this effect is expected** if we do a physical experiment of the test, the chemical species studied diffuses in the fluid, following its movement (previously we would not expect this phenomenon since we considered a *perfect* fluid). We also have an effect of ***waves*** in the fluid, which is for sure created by the Brownian movement.

Conclusion and perspectives

For this project, we studied the transport equation in both 1D and 2D cases, starting from the convection-diffusion equation and supposing the diffusion negligible. We found the analytical solution of the transport equation using the method of characteristics. We then used numerical analysis and discretization to find a solution of this equation, given some initial and boundary conditions as well as parameters for the transport fluid. The schemes studied where then developed in Python using *NumPy* and *Matplotlib*.

In 1D, we saw that the explicit upwind scheme was the numerical scheme closest to the solution but needed to fulfill the Condition-Friedrichs-Lewy condition to be stable and accurate.

For the 2D case, the most stable scheme was also the explicit upwind scheme. The other schemes and methods tested were unfortunately too unstable or wrongly programmed to perform well, and the explicit scheme did not perform well for value of length and time too high, i.e. when $L > 1m$ and $t_f > 1s$.

Finally, we studied the stochastic transport equation and the Brownian motion to create a more realistic model of the transport. Since there are no analytical solution for this equation we used the Euler-Maruyama method to numerically solve the stochastic transport equation. We are not completely sure about the results we found but it seems that the stochastic part generates some kind of diffusion or transport effect, but this could also be caused by the scheme we used.

If we were to continue this project, we could focus on several points: first, we could try to do a real experiment of the transport problem and compare it to the simulations and analytical solutions we got. Then, we could test other numerical methods to solve the transport equation, especially for the 2D case. We could try for instance the finite volume method. For the stochastic part, we could also try other methods like the Milstein method or use particle-based simulation methods like Particle in Cells (PiC). We could also study the equation in a more general way, by considering for instance non-constant velocities and anisotropic velocities for the fluid, or by considering the diffusion term of the convection-diffusion equation. The final goal would be to study the problem in 3D using the most general equation for the transport, i.e.:

$$\frac{\partial u(t,x,y,z)}{\partial t} + c_x(t,x,y,z)\frac{\partial u(t,x,y,z)}{\partial x} + c_y(t,x,y,z)\frac{\partial u(t,x,y,z)}{\partial y} + c_z(t,x,y,z)\frac{\partial u(t,x,y,z)}{\partial z} + h(t,x,y,z)\frac{\partial W_t}{\partial t} = f(t,x,y,z)$$

Annexes

Python code

Parameters.py

```python
# All physical parameters for the discretization

length: float = 5.0  # Length of the spatial domain
tf: float = 5.0  # Total time of the simulation
c: float = 3.0  # Velocity
nx: int = 100  # Number of points in the 1D space (and also 2d space)
nt: int = int(2 * c * nx)  # Number of time iterations
dx: float = length / (nx - 1)  # Space step (for x and y axes)
dt: float = tf / nt  # Time step
h: float = 3.0  # Volatility


def check_cfl(dim: int) -> None:
    """
    Verifies the Courant-Friedrichs-Lewy condition
    :param dim: Dimension (1 or 2)
    """
    k = c * dt / dx
    if dim == 2:
        k *= 2

    if k > 1:
        raise ValueError(f'CFL Condition not reached (value is {k:.2f} > 1)\n'
                         f'Change to values of c, dx or dt to verify the condition')
    else:
        print(f'CFL Verified (value is {k:.2f} <= 1)')
```

Conditions.py

```
import numpy as np
from typing import Callable
from parameters import dt, length


def test_case_1d(test_id: int) -> tuple[Callable, ...]:
    """
    Define test cases for 1D

    u0: Value when x - ct > 0

    g: Value when x - ct < 0

    f: Source term

    :param test_id: Test case's id
    :return: Functions used for the test
    """
    def test_case_1() -> tuple[Callable, ...]:
        def u0(x: float) -> float:
            return x

        def g(t: float) -> float:
            return np.sin(5 * np.pi * t)

        def f(t: float, x: float) -> float:
            return 0

        return u0, g, f

    def test_case_2() -> tuple[Callable, ...]:
        def u0(x: float) -> float:
            return np.exp(-x**2)

        def g(t: float) -> float:
            return np.sin(5 * np.pi * t)

        def f(t: float, x: float) -> float:
            return 0

        return u0, g, f

    def test_case_3() -> tuple[Callable, ...]:
        def u0(x: float) -> float:
            return np.exp(-x**2)

        def g(t: float) -> float:
            return 4 * np.abs(np.sin(5 * np.pi * t))

        def f(t: float, x: float) -> float:
            return 2

        return u0, g, f

    def test_case_4() -> tuple[Callable, ...]:
        def u0(x: float) -> float:
            return x

        def g(t: float) -> float:
            return 4 * np.abs(np.sin(8 * np.pi * t))

        def f(t: float, x: float) -> float:
            return 2

        return u0, g, f

    def test_case_5() -> tuple[Callable, ...]:
        def u0(x: float) -> float:
            return x

        def g(t: float) -> float:
            return 4 * np.abs(np.sin(8 * np.pi * t))

        def f(t: float, x: float) -> float:
            return 2
```

```
        return u0, g, f

    test_cases = {1: test_case_1(),
                  2: test_case_2(),
                  3: test_case_3(),
                  4: test_case_4(),
                  5: test_case_5()}

    if test_id in test_cases.keys():
        return test_cases.get(test_id)
    raise KeyError(f'No such test case: {test_id}')


def vectorize_test_case(test_case: tuple[Callable, ...]) -> tuple[Callable, ...]:
    """
    :param test_case: u0, g and f
    :return: Vectorized version of the test case functions
    """
    u0, g, f = test_case
    u0_vec = np.vectorize(u0, otypes=[float])
    g_vec = np.vectorize(g, otypes=[float])
    f_vec = np.vectorize(f, otypes=[float])

    return u0_vec, g_vec, f_vec


def test_case_2d(test_id: int) -> tuple[Callable, ...]:
    """
    Define test cases for 2D

    u0: Value when ||(x,y)||² - ct > 0

    g: Value when ||(x,y)||² - ct - ct < 0

    f: Source term

    :param test_id: Test case's id
    :return: Functions used for the test
    """
    def test_case_1() -> tuple[Callable, ...]:
        def u0(x: float, y: float) -> float:
            return x + y

        def g(t: float) -> float:
            return np.sin(5 * np.pi * t)

        def f(t: float, x: float, y: float) -> float:
            return 0

        return u0, g, f

    def test_case_2() -> tuple[Callable, ...]:
        def u0(x: float, y: float) -> float:
            return np.exp(-x**2 - y**2)

        def g(t: float) -> float:
            return np.sin(5 * np.pi * t)

        def f(t: float, x: float, y: float) -> float:
            return 0

        return u0, g, f

    def test_case_3() -> tuple[Callable, ...]:
        def u0(x: float, y: float) -> float:
            return np.exp(-x**2 - y**2)

        def g(t: float) -> float:
            return 4 * np.abs(np.sin(5 * np.pi * t))

        def f(t: float, x: float, y: float) -> float:
            return 2

        return u0, g, f

    def test_case_4() -> tuple[Callable, ...]:
        def u0(x: float, y: float) -> float:
            return 0
```

```
    def g(t: float) -> float | np.ndarray:
        return np.where(t < 2 * dt, np.abs(np.sin(10 * np.pi * t)), 0)

    def f(t: float, x: float, y: float) -> float:
        return 0

    return u0, g, f

test_cases = {1: test_case_1(),
              2: test_case_2(),
              3: test_case_3(),
              4: test_case_4()}

if test_id in test_cases.keys():
    return test_cases.get(test_id)
raise KeyError(f'No such test case: {test_id}')
```

Analytical.py

```
import numpy as np
from typing import Callable
from parameters import c, length, nx, nt, dt


def get_analytical_solution_1d(size: str, u0: Callable, g: Callable, f: Callable) -> np.ndarray:
    """
    :param size: 'same' for nx = nx or 'accurate' for nx = 10 * nx
    :param u0:
    :param g:
    :param f:
    :return: The analytical solution for 1D case as U,
        matrix of shape (Nt, Nx) containing all the values for each time step
    """
    if size not in ['same', 'accurate']:
        raise Exception('Size must be "accurate" or "same"')

    n = 0
    if size == 'same':
        n = nx
    if size == 'accurate':
        n = 10 * nx
    x = np.linspace(0, length, n)

    def u_1d(t: float) -> np.ndarray:
        """
        :param t: Time
        :return: Value of u everywhere on the 1d space at the time given
        """
        return np.where(x - c * t > 0, u0(x - c * t) + f(t, x) * t, g(t - x / c))

    u_mat = np.zeros((nt, n))
    u_mat[0] = np.vectorize(u0, otypes=[float])(x)

    # Compute solution at each time step
    for n in range(nt - 1):
        u_mat[n + 1] = u_1d((n + 1) * dt)

    return u_mat


def get_analytical_solution_2d(u0: Callable, g: Callable, f: Callable) -> np.ndarray:
    """
    :return: The analytical solution for 2d case as U,
        matrix of shape (Nx, Ny, Nt) containing all the values for each time step
    """
    x = np.linspace(0, length, nx)
    y = np.linspace(0, length, nx)
    x_grid, y_grid = np.meshgrid(x, y)

    def u_2d(t: float) -> np.ndarray:
        """
        :param t: Time
        :return: Value of u everywhere in space at the time given
        """
        d = x_grid ** 2 + y_grid ** 2
        return np.where(d - c * t > 0, u0(x_grid - c * t, y_grid - c * t) + f(t, x_grid, y_grid) *
t, g(t - d / c))

    u_mat = np.zeros((nx, nx, nt))
    u_mat[:, :, 0] = u0(x_grid, y_grid)

    # Compute solution at each time step
    for n in range(nt - 1):
        u_mat[:, :, n + 1] = u_2d(dt * (n + 1))

    return u_mat
```

Numerical.py

```
import numpy as np
from typing import Callable
from parameters import c, dt, dx, nx, nt, length, h


def get_numerical_solution_1d(method: str, u0_vec: Callable, g_vec: Callable, f_vec: Callable) ->
np.ndarray:
    """
    :param method: Method used (explicit-upwind, explicit-centered, implicit-upwind, implicit-
centered, crank-nicholson)
    :param u0_vec:
    :param g_vec:
    :param f_vec:
    :return: The numerical solution as U, matrix of shape (Nt, Nx) containing all the values for
each time step
    """
    if method not in ['explicit-upwind', 'implicit-upwind', 'implicit-centered', 'crank-nicholson']:
        raise Exception("Method must be explicit-upwind, implicit-upwind, implicit-centered or
crank-nicholson")

    # Initialization of the space
    x = np.linspace(0, length, nx)

    # Initialization of the matrix
    u = np.zeros((nt, nx))
    u[0] = u0_vec(x)

    # U^n+1 = A U^n + dt F^n
    if method == "explicit-upwind":
        # Computation of the matrix
        k = c * dt / dx
        a_mat = np.eye(nx) + k * (-np.eye(nx, k=0) + np.eye(nx, k=-1))

        # Scheme
        for n in range(nt - 1):
            u[n + 1] = a_mat.dot(u[n]) + dt * f_vec(n * dt, u[n])

            # Boundary conditions
            idx = np.where(x - c * (n + 1) * dt < 0)[0]
            if len(idx) != 0:
                u[n + 1, idx] = g_vec((n + 1) * dt - x[idx] / c)

    # A U^(n+1) = C U^n + dt * F^n => U^(n+1) = A^-1 * (C * U^n + dt * F^n)
    if "implicit" in method:
        # Computation of the matrices
        inv_a_mat = np.eye(nx)
        c_mat = np.eye(nx)

        if "upwind" in method:
            k = c * dt / dx
            c_mat = np.eye(nx)
            inv_a_mat = np.linalg.inv(np.eye(nx) + k * (np.eye(nx, k=0) - np.eye(nx, k=-1)))

        if "centered" in method:
            k = c * dt / (2 * dx)
            c_mat = np.eye(nx)
            inv_a_mat = np.linalg.inv(np.eye(nx) + k * (-np.eye(nx, k=1) + np.eye(nx, k=-1)))

        # Scheme
        for n in range(nt - 1):
            u[n + 1] = inv_a_mat @ (c_mat.dot(u[n]) + dt * f_vec(n * dt, u[n]))

            # Boundary conditions
            idx = np.where(x - c * (n + 1) * dt < 0)[0]
            if len(idx) != 0:
                u[n + 1, idx] = g_vec((n + 1) * dt - x[idx] / c)

    # A * U^n+1 = C * U^n + dt / 2 * F^n + dt / 2 * F^n+1
    if method == "crank-nicholson":
        k = c * dt / (2 * dx)
        inv_a_mat = np.linalg.inv(np.eye(nx) + k * (np.eye(nx, k=0) - np.eye(nx, k=-1)))
        c_mat = np.eye(nx) + k * (-np.eye(nx, k=0) + np.eye(nx, k=-1))

        for n in range(nt - 1):
            u[n + 1] = inv_a_mat @ (c_mat.dot(u[n])
                                    + dt / 2 * f_vec(n * dt, u[n])
```

```
                                    + dt / 2 * f_vec((n + 1) * dt, u[n]))

            # Boundary conditions
            idx = np.where(x - c * (n + 1) * dt < 0)[0]
            if len(idx) != 0:
                u[n + 1, idx] = g_vec((n + 1) * dt - x[idx] / c)

    return u


def get_numerical_solution_2d(method: str, u0: Callable, g: Callable, f: Callable) -> np.ndarray:
    """
    :return: The numerical solution as U, matrix of shape (Nx, Ny, Nt) containing all the values for
each time step
    """
    if method not in ['fdm-upwind', 'fdm-centered', 'rk4', 'euler-maruyama']:
        raise Exception('Method must be fdm-upwind, fdm-centered, rk4, or euler-maruyama')

    def apply_boundary(mat: np.ndarray) -> np.ndarray:
        """
        :param mat: Matrix at step n
        :return: Matrix with applied boundary conditions
        """
        new_mat = np.copy(mat)
        new_mat[0, :] = 0
        new_mat[-1, :] = 0
        new_mat[:, 0] = 0
        new_mat[:, -1] = 0
        return new_mat

    # Initialization of the space
    x = np.linspace(0, length, nx)
    y = np.linspace(0, length, nx)
    x_grid, y_grid = np.meshgrid(x, y)
    d = x_grid ** 2 + y_grid ** 2

    # Initialization of the matrix
    u = np.zeros((nx, nx, nt))
    u[:, :, 0] = u0(x_grid, y_grid)

    if method == 'rk4':
        def equation(t: float, un: np.ndarray) -> np.ndarray:
            dudx = np.zeros(un.shape)
            dudx[:, 1:] = np.roll(un[:, 1:], -1, axis=1)

            dudy = np.zeros(un.shape)
            dudy[1:, :] = np.roll(un[1:, :], -1, axis=0)

            dudx = dudx - un
            dudy = dudy - un

            return f(t, x_grid, y_grid) - c * (dudx + dudy)

        # Scheme
        for n in range(nt - 1):
            # Compute the coefficients for RK4
            u_n = u[:, :, n]
            t_n = (n + 1) * dt
            k1 = equation(t_n, u_n)
            k2 = equation(t_n + dt / 2, u_n + dt / 2 * k1)
            k3 = equation(t_n + dt / 2, u_n + dt / 2 * k2)
            k4 = equation(t_n + dt, u_n + dt * k3)

            # New value
            u[:, :, n + 1] = u[:, :, n] + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)

            # When ||(x,y)||^2 - c * t < 0
            i, j = np.where(d - c * (n + 1) * dt < 0)
            dij = (dx * i) ** 2 + (dx * j) ** 2
            u[i.tolist(), j.tolist(), n + 1] = g((n + 1) * dt - dij / c)

    if 'fdm' in method:
        func = None
        k = 0

        def upwind(un: np.ndarray) -> np.ndarray:
            """
            Upwind scheme + Null boundary conditions
```

```
        :param un:
        """
        dudx = np.zeros((nx, nx))
        dudx[:, 1:] = (un[:, 1:] - un[:, :-1]) / dx

        dudy = np.zeros((nx, nx))
        dudy[1:, :] = (un[1:, :] - un[:-1, :]) / dx

        return dudx + dudy

    def centered(un: np.ndarray) -> np.ndarray:
        """
        Centered scheme + Null boundary conditions
        :param un:
        """
        dudx = np.zeros((nx, nx))
        dudx[:, 1:-1] = (un[:, 2:] - un[:, :-2]) / (2 * dx)

        dudy = np.zeros((nx, nx))
        dudy[1:-1, :] = (un[2:, :] - un[:-2, :]) / (2 * dx)

        return dudx + dudy

    if 'upwind' in method:
        func = upwind
        k = c * dt
    if 'centered' in method:
        func = centered
        k = c * dt / 2

    # Loop
    for n in range(nt - 1):
        # Scheme + Null boundary conditions
        u_n = u[:, :, n]
        u[:, :, n + 1] = u[:, :, n] - k * func(u_n) + dt * f(n * dt, x_grid, y_grid)

        # When ||(x,y)||^2 - c * t < 0
        i, j = np.where(d - c * (n + 1) * dt < 0)
        dij = (dx * i) ** 2 + (dx * j) ** 2
        u[i.tolist(), j.tolist(), n + 1] = g((n + 1) * dt - dij / c)

if method == "euler-maruyama":
    # Define randomness
    dw = np.random.normal(loc=0.0, scale=dt, size=(nx, nx, nt))

    def grad_dx(mat: np.ndarray) -> np.ndarray:
        du_dx = mat - np.roll(mat, 1, axis=1)
        du_dx = apply_boundary(du_dx)
        return du_dx

    # u_i,j - u_i,j-1
    def grad_dy(mat: np.ndarray) -> np.ndarray:
        du_dy = mat - np.roll(mat, 1, axis=0)
        du_dy = apply_boundary(du_dy)
        return du_dy

    def upwind(mat: np.ndarray) -> np.ndarray:
        return grad_dx(mat) + grad_dy(mat)

    for n in range(nt - 1):
        u[:, :, n + 1] = (u[:, :, n]
                          - c * dt / dx * upwind(u[:, :, n])
                          + h * dw[:, :, n + 1]
                          + dt * f((n + 1) * dt, x_grid, y_grid))

        # When ||(x,y)||^2 - c * t < 0
        i, j = np.where(d - c * (n + 1) * dt < 0)
        dij = (dx * i) ** 2 + (dx * j) ** 2
        u[i.tolist(), j.tolist(), n + 1] = g((n + 1) * dt - dij / c)

        # Boundary conditions
        u[:, :, n + 1] = apply_boundary(u[:, :, n + 1])

return u


def oil in water(u0 vec: Callable, g: Callable, f: Callable, v:float=1.12 * 1e-8) -> np.ndarray:
    x = np.linspace(0, length, nx)
```

```
u = np.zeros((nt, nx))
u[0] = u0_vec(x)

for n in range(nt - 1):
    for i in range(nx):
        if i * dx - c * (n + 1) * dt < 0:
            u[n + 1, i] = g((n + 1) * dt - i * dx / c)
        else:
            uim1 = u[n, i - 1] if i > 0 else 0
            uip1 = u[n, i + 1] if i < nx - 1 else 0
            u[n + 1, i] = (u[n, i]
                           + c * dt / dx * (uim1 - u[n, i])
                           + v * dt / (dx**2) * (uim1 - 2 * u[n, i] + uip1)
                           + dt * f(n * dt, i * dx))

return u
```

Display.py

```
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import numpy as np
from parameters import length, tf, dt, nt, nx
from math import ceil

matplotlib.use('TkAgg')
matplotlib.rcParams['animation.ffmpeg_path'] = r'C:\ffmpeg\bin\ffmpeg.exe'


def show_image(u_list: list[np.ndarray], title_list=None, to_save: bool = False, path: str = "") ->
None:
    """
    Display the matrix
    :param u_list: Arrays containing all the values of U at each position, for each time step
    :param title_list : Titles of the plots
    :param to_save : True to save the figure as a PNG file
    :param path: path to the file
    """
    if title_list is None:
        title_list = len(u_list) * [""]
    # One array to plot
    if len(u_list) == 1:
        x = np.linspace(0, length, u list[0].shape[1])
        t = np.linspace(0, tf, u_list[0].shape[0])
        fig, ax = plt.subplots(figsize=(16, 9))
        im = ax.contourf(x, t, u_list[0], levels=50, cmap='viridis', vmin=np.min(u_list[0]),
vmax=np.max(u_list[0]))
        ax.set_title('Evolution of the solution for the transport equation in 1D (matrix form)\n'
                     f'{title_list[0]}')
        ax.set_xlabel('Position (x)')
        ax.set_ylabel('Time (t)')
        ax.set_aspect('equal', 'box')
        plt.colorbar(im, label='u(t,x)', shrink=0.5)
    # Multiple arrays to plot
    else:
        vmin = min([np.min(u) for u in u_list])
        vmax = max([np.max(u) for u in u_list])
        n = len(u_list)
        fig, ax = plt.subplots(nrows=n // 3 + 1, ncols=1 + n % 3, figsize=(16, 9))
        ax = ax.flatten()
        plt.suptitle('Evolution of the solution for the transport equation in 1D (matrix form)')
        for i in range(n):
            x = np.linspace(0, length, u_list[i].shape[1])
            t = np.linspace(0, tf, u_list[i].shape[0])
            im = ax[i].contourf(x, t, u_list[i], levels=50, cmap='viridis', vmin=vmin, vmax=vmax)
            ax[i].set_title(f'{title_list[i]}')
            ax[i].set_xlabel('Position (x)')
            ax[i].set_ylabel('Time (t)')
            ax[i].set_aspect('equal', 'box')
            plt.colorbar(im, label='u(t,x)', ax=ax[i])

    plt.tight_layout()

    # Save image
    if to_save and path != "":
        plt.savefig(path)
    plt.show()


def show_animation_1d(u_list: list[np.ndarray],
                      labels: list[str] = None) -> None:
    """
    Create an animation by looping on each row of the matrix
    :param u_list: Arrays containing all the values of U at each position, for each time step
    :param labels: Titles of the curves
    """
    fig, ax = plt.subplots()
    vmin, vmax = min([np.min(u) for u in u_list]), max([np.max(u) for u in u_list])

    for i in range(nt):
        ax.cla()
        for k in range(len(u_list)):
            x = np.linspace(0, length, u_list[k].shape[1])
            ax.plot(x, u_list[k][i], label=labels[k])
```

```
        ax.set_xlabel('Position x')
        ax.set_ylabel('Concentration u(t,x)')
        ax.axis((0, length, vmin, vmax))

        plt.title(f"\nt={i * dt:.2f}s")
        plt.legend(loc='upper right')
        plt.tight_layout()
        plt.pause(dt)
    plt.show()


def save_animation_1d(u_list: list[np.ndarray],
                      labels: list[str],
                      ani_name: str) -> None:
    fig, ax = plt.subplots()
    mins, maxs = min([np.min(u) for u in u_list]), max([np.max(u) for u in u_list])

    def update(i: int):
        ax.cla()
        for k in range(len(u_list)):
            x = np.linspace(0, length, len(u_list[k][i]))
            ax.plot(x, u_list[k][i], label=labels[k])
        ax.set_xlabel('Position x')
        ax.set_ylabel('Concentration u(t,x)')
        ax.axis((0, length, mins, maxs))
        plt.title(f"\nt={i * dt:.2f}s")
        plt.legend(loc='upper right')
        plt.tight_layout()

    print("Creating animation...")
    ani = FuncAnimation(fig, update, frames=range(nt), interval=dt)
    ani.save(ani_name, writer='ffmpeg', fps=30)
    print(f'Animation saved at {ani_name}')


def show_average(u_list: list[np.ndarray], title_list=None, to_save: bool = False, path: str = "") -
> None:
    """
    Plot the average value of u over time
    :param u_list: Arrays containing all the values of U at each position, for each time step
    :param title_list: Titles of the plots
    :param to_save : True to save the figure as a PNG file
    :param path: path to the file
    """
    if title_list is None:
        title_list = len(u_list) * [""]

    t = np.linspace(0, tf, nt)
    for i in range(len(u_list)):
        u_avg = np.mean(u_list[i], axis=1)
        plt.plot(t, u_avg)
        plt.title('Evolution of the average concentration in 1D')
        plt.legend(title_list)
    plt.xlabel('Time (t)')
    plt.ylabel('Average concentration')

    plt.tight_layout()

    if to_save and path != "":
        plt.savefig(path)
    plt.show()


def show_animation_2d(u_list: list[np.ndarray], title_list: list[str]) -> None:
    """
    :param u_list:
    :param title_list:
    """
    x = np.linspace(0, length, nx)
    vmin = max([np.min(u) for u in u_list])
    vmax = min([np.max(u) for u in u_list])

    if len(u_list) == 1:
        fig, ax = plt.subplots()
        for n in range(nt):
            ax.cla()
            ax.set_title(f"{title_list[0]}\n{dt * n:.2f}s")
```

```
            ax.imshow(u_list[0][:, :, n],
                      vmin=vmin,
                      vmax=vmax,
                      extent=(0, length, 0, length),
                      origin='lower')
            plt.tight_layout()
            plt.pause(dt)
    else:
        n = len(u list)
        if n == 2:
            fig, ax = plt.subplots(ncols=2, nrows=1)
        else:
            fig, ax = plt.subplots(ncols=n % 3 + 1, nrows=n // 3 + 1)
        num_plots = len(fig.get_axes())
        ax = ax.flatten()

        for i in range(n, num_plots):
            fig.delaxes(ax[i])

        for n in range(nt):
            if not plt.fignum_exists(fig.number):
                break

            for k in range(len(u_list)):
                ax[k].cla()
                ax[k].set_title(f"{title_list[k]}\n{dt * n:.2f}s | Mean : {np.mean(u_list[k][:, :,
n]):.2f}")
                ax[k].imshow(u_list[k][:, :, n],
                             vmin=vmin,
                             vmax=vmax,
                             extent=(0, length, 0, length),
                             origin='lower')
                ax[k].set_aspect('equal', 'box')
                plt.tight_layout()

            plt.pause(0.001)
    plt.show()


def save_animation_2d(u_list: list[np.ndarray], title_list: list[str], path: str) -> None:
    """
    :param u_list:
    :param title_list:
    :param path:
    """
    n = len(u_list)
    if n == 2:
        fig, ax = plt.subplots(ncols=2, nrows=1)
    else:
        fig, ax = plt.subplots(ncols=n // 3 + 1, nrows=n % 3 + 1)
    ax = ax.flatten()

    vmin = max([np.min(u) for u in u_list])
    vmax = min([np.max(u) for u in u_list])

    x = np.linspace(0, length, nx)

    def update(i: int):
        for k in range(len(u_list)):
            ax[k].cla()
            ax[k].imshow(u_list[k][:, :, i], vmin=vmin, vmax=vmax, extent=(0, length, 0, length),
origin='lower')
            ax[k].set_title(f"{title_list[k]}\nt={i * dt:.2f}s | Mean : {min(np.mean(u_list[k][:, :,
i]), 999):.2f}")
            ax[k].set_aspect('equal', 'box')
            plt.tight_layout()

    print('Creating animation...')
    animation = FuncAnimation(fig,
                              update,
                              frames=range(nt),
                              interval=dt)
    animation.save(path, writer='ffmpeg', fps=30)
    print(f"Animation saved at {path}")
```

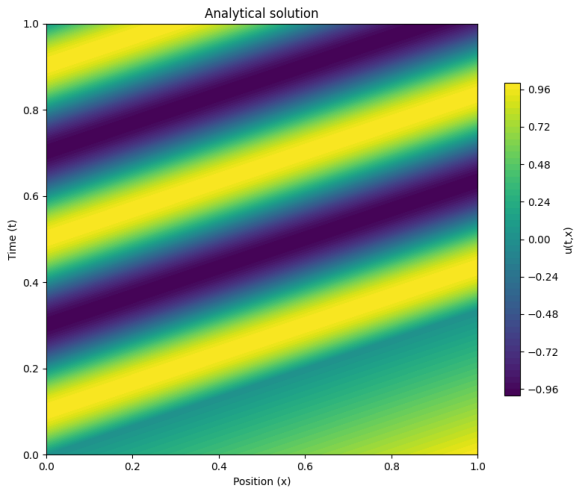
Wiener.py

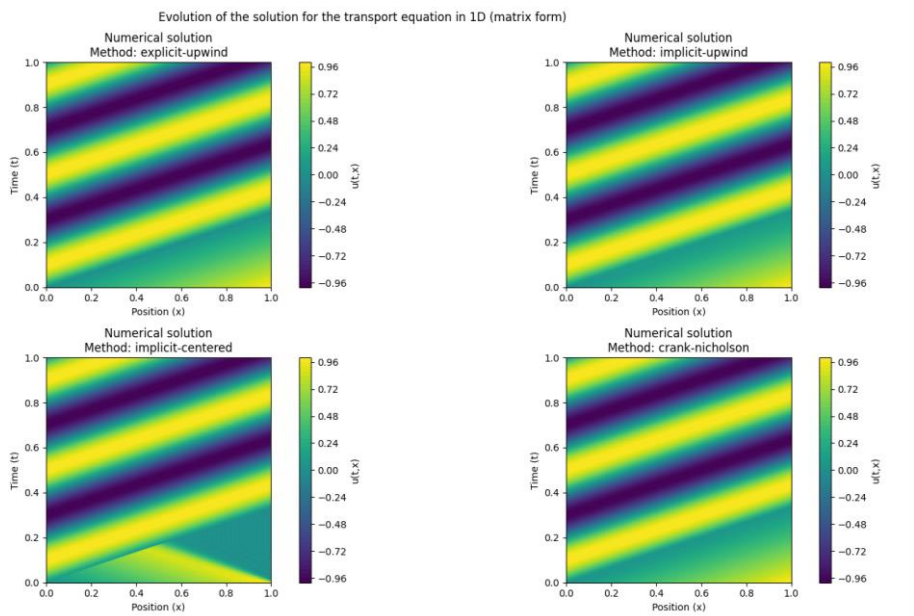
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
from matplotlib.animation import FuncAnimation


def wiener_process(mu: float, sigma2: float, nb_iter: int, x0: float = 0.0, y0: float = 0.0) ->
np.ndarray[float]:
    """
    Describes a Wiener process
    :param mu: Mean
    :param sigma2: Variance
    :param nb_iter: Number of steps
    :param x0: Initial position of the particle along the x-axis
    :param y0: Initial position of the particle along the y-axis
    :return: Matrix of size (nb_iter, 2)
    """
    # First define the displacement at each step using the normal distribution
    random_dsp = np.random.normal(loc=mu, scale=sigma2, size=(nb_iter - 1, 2))

    # Matrix containing the position of the particle at each step
    p = np.vstack(([x0, y0], random_dsp))

    # We do the cumulative sum to know how the position of the particle evolves
    p = np.cumsum(p, axis=0, dtype=float)

    return p


def plot_wiener(p: np.ndarray[float], mu: float, sigma2: float) -> None:
    """
    Plot the brownian motion of a particle
    :param p: Matrix containing the brownian motion
    :param mu: Mean
    :param sigma2: Variance
    """
    cmap = matplotlib.cm.winter
    plt.plot(p[:, 0], p[:, 1], c='black', linewidth=1, linestyle='--')
    plt.scatter(p[:, 0], p[:, 1], c=np.linalg.norm(p, axis=1), cmap=cmap)
    plt.text(p[0, 0], p[0, 1], '($x_0$,$y_0$)')
    plt.text(p[-1, 0], p[-1, 1], '($x_f$,$y_f$)')
    plt.title(f"Brownian Motion\n$\mu$={mu} | $\sigma^2$={sigma2}")

    plt.tight_layout()
    plt.show()


def animate_wiener(p: np.ndarray, mu: float, sigma2: float) -> None:
    """
    Animation of the brownian movement
    :param p: Matrix containing the brownian motion
    :param mu: Mean
    :param sigma2: Variance
    """
    fig, ax = plt.subplots()
    cmap = matplotlib.cm.winter
    colors = np.linalg.norm(p, axis=1)

    def update(i):
        if i > len(p):
            ax.plot(p[:, 0], p[:, 1], c='black', linewidth=1, linestyle='--')
            ax.scatter(p[:, 0], p[:, 1], c=np.linalg.norm(p, axis=1), cmap=cmap)
            ax.text(p[0, 0], p[0, 1], '($x_0$,$y_0$)')
            ax.text(p[-1, 0], p[-1, 1], '($x_f$,$y_f$)')
        else:
            ax.plot(p[:i, 0], p[:i, 1], c='black', linewidth=1, linestyle='--')
            ax.scatter(p[:i, 0], p[:i, 1], c=colors[:i], cmap=cmap)
            ax.text(p[0, 0], p[0, 1], '($x_0$,$y_0$)')

        ax.set_title(f"Brownian Motion\n$\mu$={mu} | $\sigma^2$={sigma2}")
        plt.tight_layout()

    print("Creating animation...")
    animation = FuncAnimation(fig,
                              update,
                              frames=len(p) + 10,
```

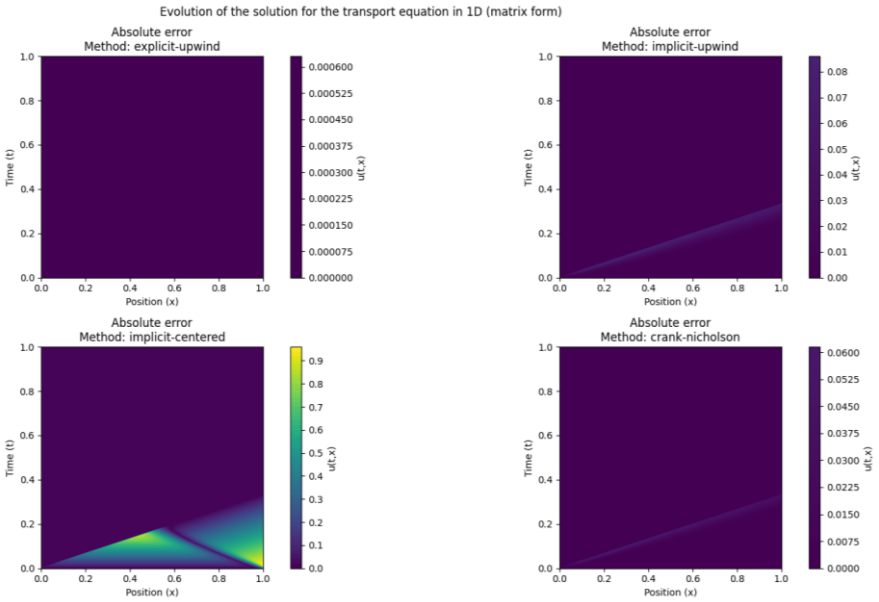
```
                                interval=0.001)
    animation.save(filename='brownian_motion.gif', writer='ffmpeg', fps=10)
    print("Animation saved !")


def plot_normal(mu: float, sigma2: float) -> None:
    """
    Plot the normal distribution used
    :param mu: Mean
    :param sigma2: Variance
    """

    def gaussian(x):
        return 1 / np.sqrt(sigma2 * 2 * np.pi) * np.exp(-(x - mu) ** 2 / (2 * sigma2))

    xl = 5 * sigma2 ** 0.5
    n = 10_000
    x = np.linspace(-xl, xl, n)
    y = gaussian(x)

    rd = np.random.normal(loc=mu, scale=sigma2, size=(n,))
    plt.title(f"Normal distribution\n$\mu$={mu} | $\sigma^2$={sigma2}")
    plt.hist(rd, bins=100, edgecolor='black', density=True)
    plt.plot(x, y, c="red")
    plt.show()


def main():
    matplotlib.use('TkAgg')
    matplotlib.rcParams['animation.ffmpeg_path'] = r'C:\ffmpeg\bin\ffmpeg.exe'

    mu = 0.0
    sigma2 = 1.0
    nb_iter = 100
    x0, y0 = 0, 0

    p = wiener_process(mu, sigma2, nb_iter, x0, y0)

    plot_normal(mu=mu, sigma2=sigma2)
    plot_wiener(p=p, mu=mu, sigma2=sigma2)
    animate_wiener(p=p, mu=0.0, sigma2=1.0)


if __name__ == '__main__':
    main()
```

1D Space-Time representation of test cases

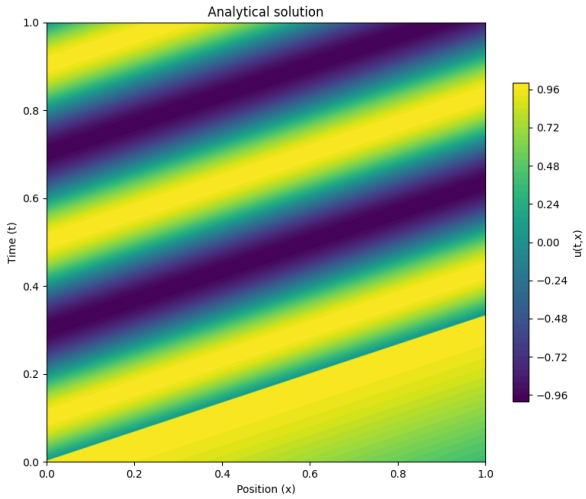
Test case 1

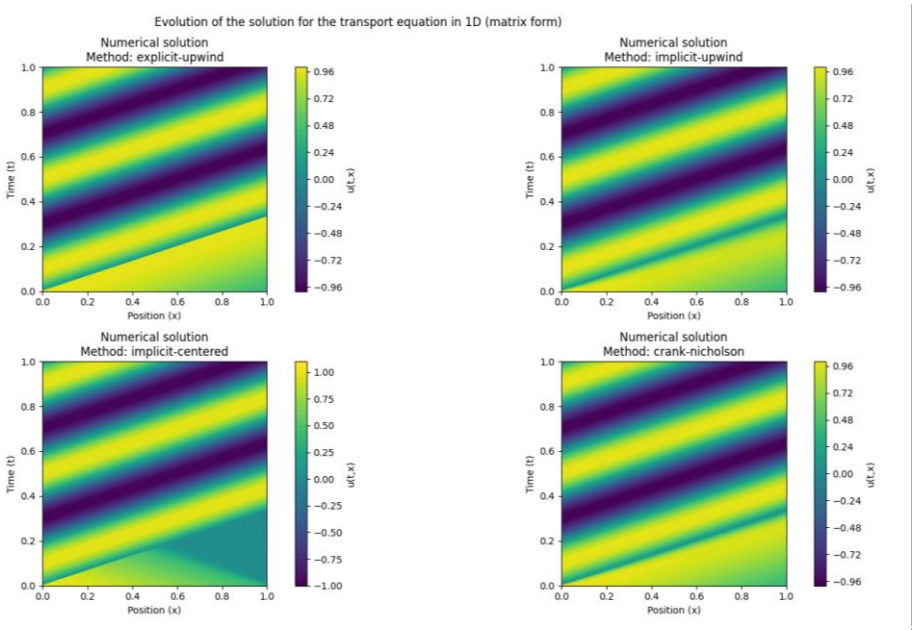


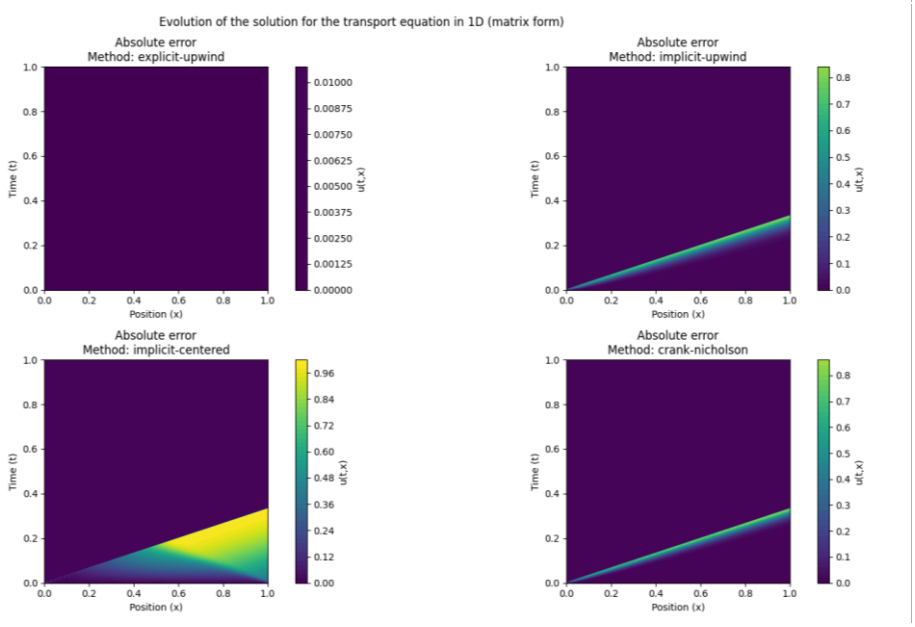



Evolution of the solution for the transport equation in 1D (matrix form)
Absolute error
Method: explicit-upwind
Absolute error
Method: implicit-upwind
Absolute error
Method: implicit-centered
Absolute error
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)

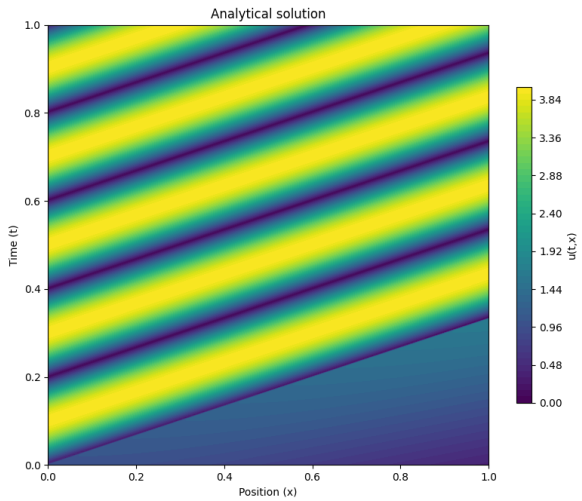
Test case 2

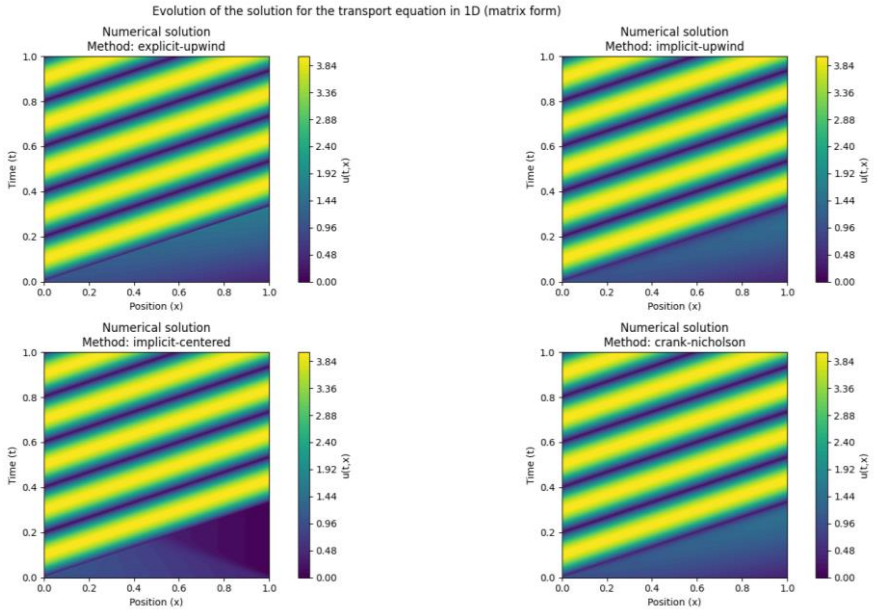

Analytical solution
Time (t)
Position (x)
u(t,x)

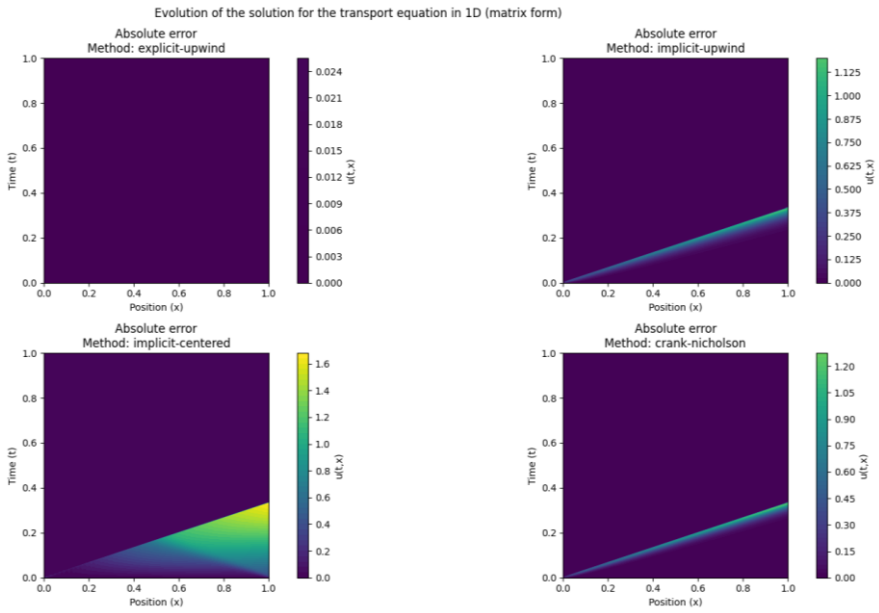

Evolution of the solution for the transport equation in 1D (matrix form)
Numerical solution
Method: explicit-upwind
Numerical solution
Method: implicit-upwind
Numerical solution
Method: implicit-centered
Numerical solution
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)


Evolution of the solution for the transport equation in 1D (matrix form)
Absolute error
Method: explicit-upwind
Absolute error
Method: implicit-upwind
Absolute error
Method: implicit-centered
Absolute error
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)

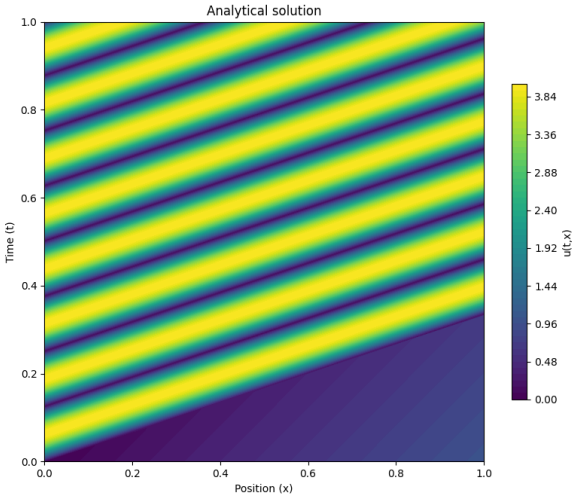
Test case 3

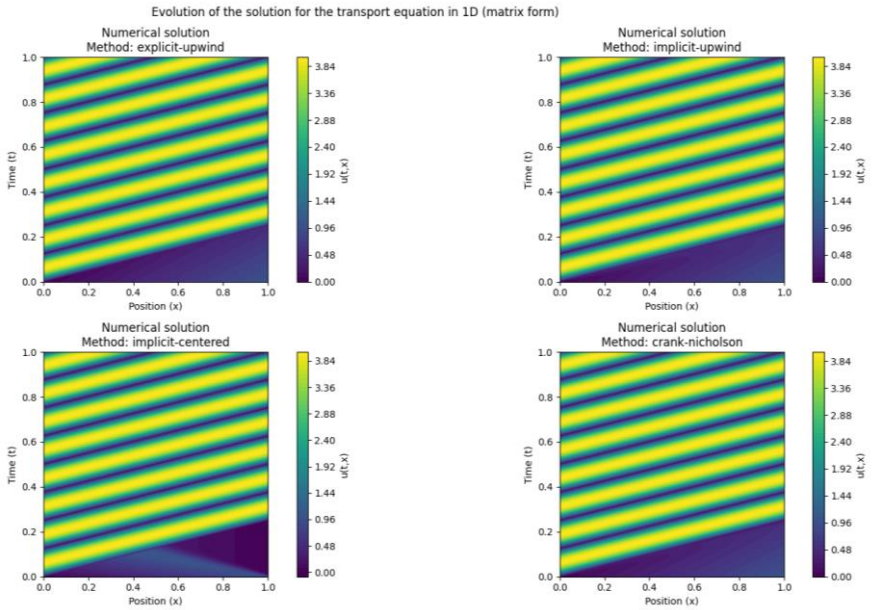

Analytical solution
Time (t)
Position (x)
u(t,x)

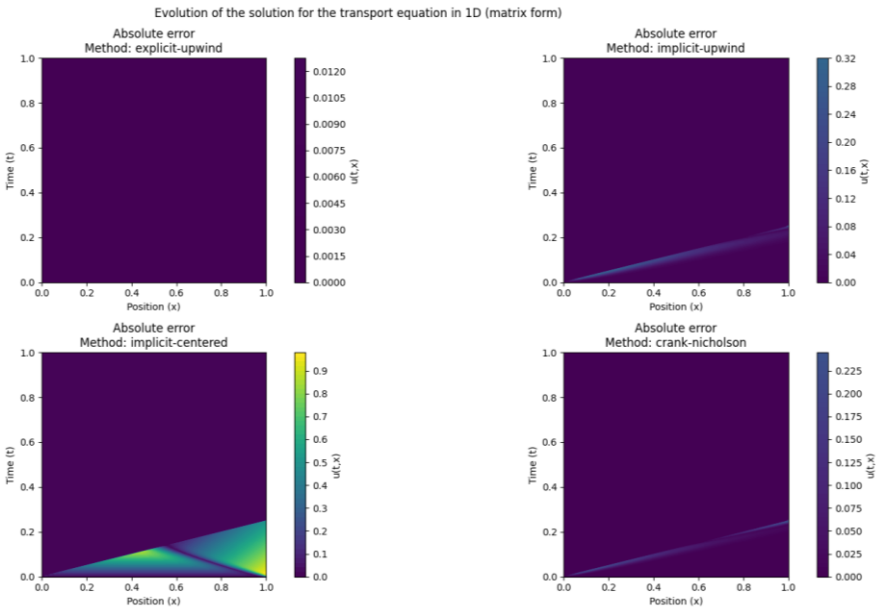

Evolution of the solution for the transport equation in 1D (matrix form)
Numerical solution
Method: explicit-upwind
Numerical solution
Method: implicit-upwind
Numerical solution
Method: implicit-centered
Numerical solution
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)


Evolution of the solution for the transport equation in 1D (matrix form)
Absolute error
Method: explicit-upwind
Absolute error
Method: implicit-upwind
Absolute error
Method: implicit-centered
Absolute error
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)

Test case 4


Analytical solution
Time (t)
Position (x)
u(t,x)


Evolution of the solution for the transport equation in 1D (matrix form)
Numerical solution
Method: explicit-upwind
Numerical solution
Method: implicit-upwind
Numerical solution
Method: implicit-centered
Numerical solution
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)


Evolution of the solution for the transport equation in 1D (matrix form)
Absolute error
Method: explicit-upwind
Absolute error
Method: implicit-upwind
Absolute error
Method: implicit-centered
Absolute error
Method: crank-nicholson
Time (t)
Position (x)
u(t,x)

Sources

About the transport equation

[1] Convection-diffusion equation
[2] Péclet number
[3] Method of characteristics

About the numerical solutions

[4] Courant-Friedrichs-Lewy condition
[5] Finite difference method
[6] Upwind scheme
[7] Simulation numérique du phénomène d'advection et de diffusion de la concentration d'un polluant dans un écoulement permanent bidimensionnel par la méthode des volumes finis

The oil in water case

[8] Transport de fluides miscibles à propriétés physiques variables en cellule Hele-Shaw. Comparaisons entre simulations numériques et mesures par LIF

The stochastic transport equation

[9] Stochastic partial differential equation
[10] Euler-Maruyama method
[11] Milstein method

Other

[12] Github link for this project code
[13] Modélisation tridimensionnelle du transport de polluant dans les écoulements à surface libre
[14] Transport de polluants dans les eaux souterraines
[15] Analyse et modélisation numérique du transport de polluants émergents et de métaux traces dans un cours d'eau, en aval d'une station d'épuration des eaux usées
[16] Transfert d'une pollution soluble dans un aquifère par une approche multi-agents
[17] Etude expérimentale du transport d'un polluant actif en canal rectangulaire